Estructuras de programación

Objetivos

- Comprender la diferencia entre análisis léxico, sintaxis y semántica.
- Identificar los componentes léxicos de Java.
- Construir y evaluar expresiones matemáticas y lógicas.
- Comprender las reglas de promoción.
- Utilizar métodos de la biblioteca estándar de Java.
- Reconocer los distintos tipos de sentencias.
- Entender el concepto de flujo de control.
- Desarrollar algoritmos iterativos.
- Realizar trazas de ejecución.
- Identificar errores frecuentes.

Contenido

Introducción 4

Análisis léxico y componentes del lenguaje 4
  Concepto de Token y separador 5
  Literales 6
  Palabras reservadas 14

Expresiones y operadores 16
  Operadores 17
  Evaluación en cortocircuito 20
  Precedencia y asociatividad 21

Conversión de tipos 23
  Reglas de promoción 24
  Conversión explícita 25
  Clases envolventes 26

Uso de la biblioteca estándar 27
  La clase Math 28
  La biblioteca como herramienta de programación 32

Instrucciones 32
  Sentencias 33
  Bloques y ámbito 35

Flujo de control 37
  Concepto de flujo de control 38
  Estructuras de control 39

Secuencia 39

Selección 41
  Selección simple 41
  Selección compuesta 43
  Selección múltiple 46
    Anidación 46
    Selección encadenada 47
  Selección múltiple con switch 47

Repetición 54
  Sentencia while 56
  Bucles controlados por condición 60
  Bucles controlados por contador 64
  Operadores de asignación 66

- Sentencia for 66
- Diseño de bucles 71

Depuración y verificación 74

- Trazas de ejecución 78
- Errores comunes 82

Introducción

Hasta este punto de nuestro aprendizaje, hemos comprendido que un programa de ordenador es, en esencia, un algoritmo escrito para ser ejecutado por una máquina. En el tema anterior, exploramos los cimientos: aprendimos a declarar variables, a reconocer tipos de datos primitivos y a utilizar la biblioteca estándar de Java para realizar operaciones de entrada/salida básicas. Sin embargo, si analizamos los programas que hemos construido, observaremos que todos comparten una característica común: son puramente lineales. El ordenador se limita a ejecutar una instrucción tras otra, en el estricto orden físico en que fueron escritas.

No obstante, la programación en el «mundo real» y la resolución de problemas técnicos complejos en entornos productivos requieren una sofisticación mucho mayor. El pensamiento computacional no es una simple sucesión de pasos, sino un tejido o «urdimbre» donde cada hebra lógica debe entrelazarse con precisión. Para lograr esto, un lenguaje de programación debe ser entendido como un conjunto de reglas, símbolos y palabras que nos permiten modelar la realidad.

El proceso por el cual el ordenador interpreta nuestras intenciones comienza con el análisis léxico. Cuando el compilador recibe nuestro código fuente, su primera tarea es identificar elementos con significado propio. Como sucede en el lenguaje natural, no podemos formar una oración coherente amontonando palabras: en los lenguajes de programación combinamos estos componentes léxicos para formar expresiones. Algo como n + 1, por sí solo, es solo un valor latente; para que sea algo capaz de realizar una tarea, debe integrarse en una estructura que le dé sentido.

La verdadera potencia del software reside en la capacidad de romper la linealidadd el código para establecer un flujo de control. Este define el orden en que las sentencias se ejecutan realmente durante la actividad de la aplicación. Para ilustrarlo, podemos usar la analogía de los músicos: ir tocando las notas según indica el director equivale a una estructura secuencial; sin embargo, podremos tomar una decisión basada en una condición o, quizás, que una serie de acordes se repitan.

Profundizaremos aquí en cómo construir expresiones y conoceremos la selección y la iteración, herramientas que nos permitirán empezar a desarrollar algoritmos inteligentes, capaces de tomar decisiones autónomas y procesar información de forma eficiente, sentando así las bases necesarias antes de adentrarnos en los fundamentos de la programación orientada a objetos.

Análisis léxico y componentes del lenguaje

Como hemos señalado, la construcción de un programa comienza con la escritura de **código fuente**, una tarea que realizamos utilizando un editor y siguiendo las reglas gramaticales del lenguaje. Sin embargo, para que este texto cobre vida y se convierta en una serie de acciones ejecutables, debe pasar por un proceso de transformación riguroso. El primer paso de este viaje técnico es el **análisis léxico**, una fase en la que el compilador actúa como un lector meticuloso que descompone la secuencia ininterrumpida de caracteres que le entregamos en piezas individuales dotadas de significado.

En el lenguaje natural, cuando leemos una frase, nuestro cerebro identifica automáticamente dónde termina una palabra y comienza la siguiente, reconociendo sustantivos, verbos y adjetivos. En el ámbito de la programación, el ordenador realiza una operación análoga. No percibe el código como un conjunto de instrucciones complejas de un solo vistazo, sino como un flujo de símbolos pertenecientes al alfabeto; en el caso de Java, *Unicode*. La labor del analizador léxico consiste en agrupar estos símbolos en unidades atómicas, que son los componentes básicos que conforman la estructura del lenguaje.

Esta etapa es fundamental porque establece los cimientos sobre los que se construirá la lógica del programa, permitiendo ensamblar estos bloques en estructuras sintácticamente correctas. Si el analizador léxico encuentra un símbolo inesperado o una combinación de caracteres que no encaja en las categorías permitidas, el proceso se detendrá antes incluso de intentar comprender la intención del algoritmo. Por lo tanto, antes de profundizar en cómo tomar decisiones o repetir procesos, es esencial comprender la naturaleza de estos elementos mínimos y los mecanismos que permiten al compilador distinguirlos unos de otros.

Analizador léxico: Fase del compilador que descompone el flujo continuo de caracteres del código fuente en unidades atómicas con significado (*tokens*).

Concepto de Token y separador

Dentro del proceso de análisis léxico, el compilador debe ser capaz de reconocer los componentes básicos que integran el código fuente para poder interpretarlos adecuadamente. Estos componentes mínimos, que no pueden ser descompuestos en partes más pequeñas sin perder su sentido, reciben el nombre de ***tokens*** o elementos atómicos. En la gramática de un lenguaje de programación como Java, los *tokens* son el equivalente a las palabras en una oración del lenguaje natural; cada uno de ellos cumple una función específica, ya sea representar un valor constante, nombrar una variable o indicar una operación matemática.

Para que el analizador léxico pueda distinguir estos elementos dentro de la secuencia ininterrumpida de caracteres que escribimos, el lenguaje emplea unos símbolos especiales denominados **separadores**. Un separador es cualquier carácter o marca que indica de manera inequívoca dónde termina un *token* con significado propio y dónde comienza el siguiente. En este sentido, la programación vuelve a imitar a la lengua: así como usamos espacios para separar palabras en un texto escrito para que no se conviertan en una masa de letras incomprensible, Java utiliza los separadores para evitar que los *tokens* se fusionen.

Token: Unidad mínima o componente atómico que posee significado propio dentro de la gramática del lenguaje de programación.

Separador: Carácter o marca que delimita inequívocamente el fin de un *token* y el inicio del siguiente.

Los elementos que actúan como separadores en Java son los siguientes:

- **Espacio en blanco:** es el separador más común y sencillo, utilizado de la misma forma que en la escritura convencional.
- **Tabulador:** actúa de forma equivalente al espacio en blanco desde el punto de vista del compilador.
- **Retorno de carro o salto de línea:** indica el fin de una línea física, pero léxicamente tiene el mismo valor que un espacio. El compilador es capaz de tratar un programa completo escrito en una sola línea o con cada palabra en una línea distinta con el mismo resultado técnico, ya que sustituye cualquier secuencia de estos separadores por un único espacio en blanco[1].

Una vez que el analizador léxico ha hecho uso de estos separadores para aislar los *tokens*, estos se clasifican en diferentes categorías según su naturaleza, como los literales, los identificadores, los operadores y las palabras reservadas. Solo si el ensamblaje de estos bloques atómicos sigue las reglas de la sintaxis, el compilador podrá avanzar hacia la fase de traducción y ejecución del programa.

Literales

Un **literal** —o valor literal—, como ya vimos en el capítulo anterior, es la representación explícita y constante de un valor concreto directamente escrito en el código fuente de un programa. A través de los literales, el programador especifica valores fijos para los tipos de datos primitivos, el tipo de datos `String` y la referencia especial `null`.

Desde el punto de vista del análisis léxico, los literales son identificados por el compilador como *tokens* atómicos. Esto significa que constituyen unidades de información indivisibles con significado propio dentro de la gramática del lenguaje. A diferencia de lo que ocurre con un identificador o una variable, cuyo contenido o estado puede fluctuar dinámicamente durante el ciclo de vida de la ejecución, el valor representado por un literal queda prefijado de manera inmutable desde la fase de compilación.

Literal: *Token* que representa la expresión inalterable de un valor constante de un tipo de datos primitivo, de un objeto `String` o de la referencia nula `null` dentro del código fuente.

---

[1] Como vimos en el capítulo anterior, esto no es exactamente así cuando se trata de una cadena, por ejemplo.

Dentro de la especificación sintáctica de Java, los literales se clasifican rigurosamente según la categoría de datos a la que corresponden, obedeciendo a un conjunto estricto de convenciones léxicas para su escritura. Plantilla sintáctica de declaración de literal:

```
Literal:
  IntegerLiteral
  FloatingPointLiteral
  BooleanLiteral
  CharacterLiteral
  StringLiteral
  NullLiteral
```

Literales enteros

De forma predeterminada, cuando el analizador léxico procesa un literal numérico compuesto únicamente por dígitos (sin coma ni punto fraccionario), el compilador deduce que se trata de un valor de tipo `int` codificado en 32 bits. La representación convencional se realiza en decimal, mediante una secuencia de dígitos del `0` al `9`. Un literal decimal no debe comenzar con el dígito `0` (salvo el propio número cero).

```
IntegerLiteral:
  DecimalNumeral

DecimalNumeral:
  0
  NonZeroDigit [Digits]

NonZeroDigit:
  (one of)
  1 2 3 4 5 6 7 8 9

Digits:
  Digit
  Digit Digits

Digit:
  0
  NonZeroDigit
```

Esta sucesión de reglas merece una pequeña explicación. Un literal de tipo entero, *`IntegerLiteral`*, es bien el número cero o un dígito distinto de cero, *`NonZeroDigit`*, seguido, opcionalmente, de dígitos, *`Digits`*. Estos dígito, a su vez, pueden ser un dígito *`Digit`* o un dígito seguido de dígitos. Por último, un dígito es el número cero o un número distinto de cero. La especificación lo que nos dice es que un número entero puede tener un número indeterminado de dígitos pero que no puede comenzar por cero, salvo que sea el cero.

A partir de Java 7, se permite intercalar el carácter de subrayado (`_`) entre los dígitos de cualquier literal numérico para mejorar la legibilidad del código fuente (por ejemplo, 1_000_000). Estos caracteres son ignorados por el analizador léxico durante la fase de compilación y no afectan al valor asignado.

```
Digits:
  Digit
  Digit [DigitsAndUnderscores] Digit

DigitsAndUnderscores:
  DigitOrUnderscore {DigitOrUnderscore}

DigitOrUnderscore:
  Digit
  _
```

Además del formato convencional, en decimal, Java permite representar literales enteros mediante otros tres sistemas de numeración:

- **Hexadecimal (base 16):** se denota mediante el prefijo `0x` o `0X`, aceptando dígitos del `0` al `9` y letras de la `a` a la `f` (tanto en mayúsculas como en minúsculas). Un ejemplo es el número `26`: `0X1A`.
- **Octal (base 8):** se especifica anteponiendo un cero (`0`) como prefijo al número, seguido exclusivamente por dígitos comprendidos entre el `0` y el `7` (por ejemplo, `012`, que es el número `10`).
- **Binario (base 2):** permite la notación explícita mediante el prefijo `0b` o `0B`, empleando únicamente los dígitos `0` y `1` (por ejemplo, el número `10` se representa como `0b1010`[2]).

```
IntegerLiteral:
  DecimalNumeral
  HexNumeral
  OctalNumeral
  BinaryNumeral

HexNumeral:
  0 x HexDigits
  0 X HexDigits

HexDigits:
  HexDigit
  HexDigit [HexDigitsAndUnderscores] HexDigit

HexDigit:
  (one of)
  0 1 2 3 4 5 6 7 8 9 a b c d e f A B C D E F
```

---

[2] La capacidad de escribir literales en notación binaria directa mediante el prefijo `0b` o `0B` fue introducida formalmente en la versión Java SE 7 mediante la propuesta JEP 100.

```
HexDigitsAndUnderscores:
  HexDigitOrUnderscore {HexDigitOrUnderscore}

HexDigitOrUnderscore:
  HexDigit
  _

OctalNumeral:
  0 OctalDigits
  0 OctalDigits

OctalDigits:
  OctalDigit
  OctalDigit [OctalDigitsAndUnderscores] OctalDigit

OctaDigit:
  (one of)
  0 1 2 3 4 5 6 7

OctalDigitsAndUnderscores:
  OctalDigitOrUnderscore {OctalDigitOrUnderscore}

OctalDigitOrUnderscore:
  OctalDigit
  _

BinaryNumeral:
  0 b BinaryDigits
  0 B BinaryDigits

BinaryDigits:
  BinaryDigit
  BinaryDigit [BinaryDigitsAndUnderscores] BinaryDigit

BinaryDigit:
  (one of)
  0 1

BinaryDigitsAndUnderscores:
  BinaryDigitOrUnderscore {BinaryDigitOrUnderscore}

BinaryDigitOrUnderscore:
  BinaryDigit
  _
```

Si se requiere que el literal sea interpretado por el compilador como un entero de precisión extendida de 64 bits (`long`), es obligatorio añadir el sufijo `L` o `l` al final de la secuencia de dígitos[3].

```
IntegerLiteral:
  DecimalNumeral [IntegerTypeSuffix]
  HexNumeral [IntegerTypeSuffix]
  OctalNumeral [IntegerTypeSuffix]
  BinaryNumeral [IntegerTypeSuffix]

IntegerTypeSuffix:
  (one of)
  l L
```

Internamente, la máquina virtual Java (JVM) almacena todos los valores enteros utilizando la representación binaria en **complemento a dos con signo**[4]. Este formato fija los límites inferior y superior de los rangos numéricos que cada tipo de entero puede albergar en memoria (por ejemplo, entre -2^{31} y $2^{31}-1$ para el tipo `int`).

```
int contador = 100;           // Decimal
int mascaraHex = 0xFF;        // Hexadecimal (255)
int patronBits = 0b1100;      // Binario (12)
long poblacion = 8000000000L; // Literal de 64b
```

Literales de punto flotante

Los literales de punto flotante se emplean para expresar números reales. Por defecto, la gramática de Java interpreta cualquier literal con punto decimal o notación exponencial como un valor de tipo `double` (64 bits), garantizando un grado elevado de precisión matemática.

Existen dos formas estándar de escribir literales de punto flotante:

- **Notación decimal:** utiliza el carácter punto (`.`) para separar explícitamente la parte entera de la fraccionaria (por ejemplo, `3.14159` o `5`).
- **Notación científica o exponencial:** añade la letra `e` o `E` seguida de un número entero (positivo o negativo) que representa un exponente o potencia de diez (por ejemplo, `1.5e3` para representar 1.5×10^3, es decir, `1500.0`, o `2.5E-4` para 2.5×10^{-4}).

```
FloatingPointLiteral:
  DecimalFloatingPointLiteral [FloatTypeSuffix]
```

---

[3] Se recomienda enfáticamente el uso de la letra `L` mayúscula para evitar confusiones visuales con el número uno (1).

[4] En la representación en complemento a dos, el bit más significativo, el primero, es 1 cuando el número es negativo. Los números negativos se calculan cambiando los unos por ceros y los ceros por unos, sumando uno al resultante.

```
DecimalFloatingPointLiteral:
  Digits . [Digits] [ExponentPart]
  . Digits [ExponentPart]
  Digits ExponentPart
  Digits [ExponentPart]

ExponentPart:
  ExponentIndicator SignedInteger

ExponentIndicator:
  (one of)
  e E

SignedInteger:
  [Sign] Digits

Sign:
  (one of)
  + -

FloatTypeSuffix:
  (one of)
  f F e E
```

A partir de las reglas anteriores podemos ver que un número en punto flotante puede empezar por punto cuando la parte entera sea cero. Cuando sea preciso declarar un literal de precisión simple de 32 bits (`float`) —para cumplir con los requerimientos de un método, por ejemplo—, se debe añadir el sufijo `f` o `F` al final del número.

```
double pi = 3.1415926535;   // Tipo double por defecto
double masa = 5.972e24;     // Notación científica (double)
float gravedad = 9.81f;     // Sufijo f para forzar el tipo float
```

Supongamos que nuestra máquina, en la que cada ubicación de memoria tiene el mismo tamaño, permite representar los números mediante seis dígitos y un signo. Cuando definimos una variable o constante, la ubicación asignada consta de seis dígitos y un signo; si el valor es entero, la interpretación del número almacenado en esa ubicación es sencilla: puedo almacenar valores desde el -999999 hasta el +999999; el cero tiene dos representaciones. Nuestra precisión es seis dígitos y los números en ese rango pueden ser representados de forma exacta.

Supongamos que los dos dígitos de más a la izquierda nos permiten representar un exponente. La representación +561234, por ejemplo, es en realidad el número 1234×10^{56}. De hecho, el rango de números que ahora podemos representar es mucho mayor: desde -9999×10^{99} hasta $+9999 \times 10^{99}$. Sin embargo, la precisión ahora es de solo cuatro dígitos; es decir, solo los números de cuatro dígitos pueden representarse con exactitud en nuestro sistema. ¿Qué sucede con los números con más dígitos? Los cuatro dígitos de la izquierda se representan correctamente, y los dígitos de la derecha, o dígitos menos significativos, se pierden (se asume que son 0). Por ejemplo, 1000000 puede representarse con exactitud, pero 4932416 no, porque nuestro esquema de codificación nos limita a cuatro dígitos significativos.

Precisión: El máximo número de dígitos significativos.

Dígitos significativos: Desde el primer dígito distinto de cero a la izquierda hasta el último dígito distinto de cero a la derecha (más cualquier dígito cero que sea exacto).

Para extender nuestro esquema de codificación y representar números de punto flotante, debemos poder representar exponentes negativos. Dado que nuestro esquema no incluye un signo para el exponente, vamos a modificarlo ligeramente: el signo existente se convierte en el signo del exponente y añadimos un signo a la izquierda para representar el signo del número.

Ahora podemos representar con precisión, con cuatro dígitos, todos los números entre -9999×10^{99} y $+9999 \times 10^{99}$. Añadir exponentes negativos a nuestro esquema nos permite representar fracciones tan pequeñas como 1×10^{-99}. Nuestra precisión sigue siendo de cuatro dígitos. Los números 0.1032, 5.406 y 1000000 se pueden representar con exactitud. El número 476.0321, sin embargo, tiene siete cifras significativas, pero se representa como 476.0; ese 0.0321 no se puede representar en nuestro sistema[5].

La codificación interna sigue la norma **IEEE 754** (formato binario de punto flotante para ordenadores), estructurando el valor en tres campos binarios diferenciados: bit de signo, mantisa y exponente. En el caso de los vaores de tipo `float`, el bit de signo es +1 o -1, la mantisa es un número entero positivo menor que 2^{24} y el exponente en un número entre -126 y 127, incluidos. En el caso de los valores de tipo `double`, la mantisa en un número entero positivo menor que 2^{53} y el exponente en un número entre -1022 y 1023, incluidos.

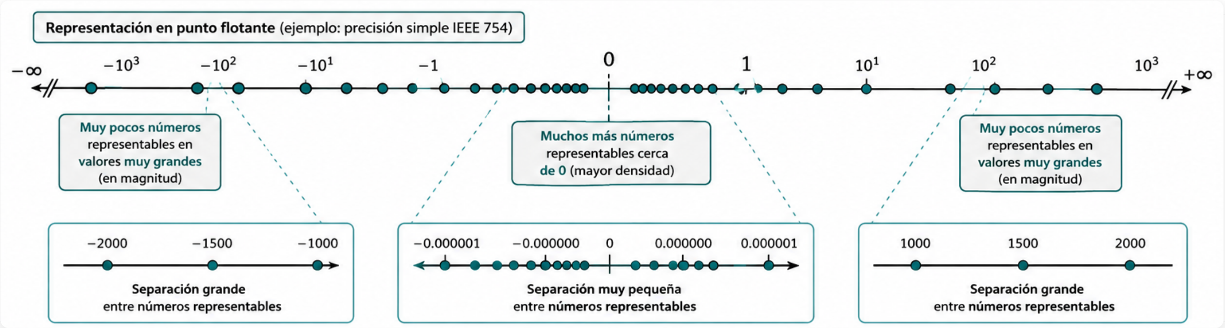
El nombre *punto flotante* hace referencia a que el *punto decimal* puede moverse. En nuestro modelo, cada número se almacena en cuatro dígitos; si el dígito que no es cero se almacena en la posición de más a la izquierda, el exponente se ajusta de la manera correspondiente: por ejemplo, 1000000 podríamos almacenarlo como +060001 pero lo almacenamos como +031000. Este formato proporciona la máxima precisión posible, denominándose formato normalizado. En el modelo que usa la JVM, además, solo puede haber un único dígito distinto de cero a la izquierda del punto decimal, lo que nos permite aumentar el número de dígitos de la mantisa en uno, por lo que la preción no es de 24 o 53 bits: pasa a ser 25 y 54. Esto es debido a que sabemos con certeza que el primer dígito siempre es 1 -al representarlo en base 2- y la máquina no lo almacena.

Es evidente que podemos representar valores extremadamente amplios o infinitesimales, asumiendo las limitaciones de redondeo y precisión propias de la aritmética de punto flotante. Sin embargo, con la explicación anterior, podríamos pensar que el cero no puede representarse. La realidad es que existen patrones para designar números específicos.

---

[5] La JVM realiza redondeo en lugar de truncamiento al descartar los dígitos sobrantes. Si asumimos que tiene cuatro cifras significativas, el redondeo almacenaría 476.0321 como 476.0, pero almacenaría 476.0823 como 476.1.

Y aunque pueda parecer otra cosa, hay que tener en cuenta que el número máximo de valores diferentes que la notación en punto flotante puede representar es 2^{32} o 2^{64} números: lo que hemos hecho es expandir esos números en un rango mayor. Además, los números que podemos representar no están espaciados de la misma manera entre ellos, como los números en punto fijo. Los posibles valores están más próximos cerca del origen y más separados cuanto más alejados.


Representación en punto flotante (ejemplo: precisión simple IEEE 754)
−∞
-10^3
-10^2
-10^1
−1
0
1
10^1
10^2
10^3
+∞
Muy pocos números representables en valores muy grandes (en magnitud)
Muchos más números representables cerca de 0 (mayor densidad)
Muy pocos números representables en valores muy grandes (en magnitud)
−2000
−1500
−1000
Separación grande entre números representables
−0.000001
−0.000000
0
0.000000
0.000001
Separación muy pequeña entre números representables
1000
1500
2000
Separación grande entre números representables

Otros literales

Para la representación explícita de valores no numéricos o de estructuras complejas, el lenguaje provee la siguiente gama de literales:

- **Literales booleanos:** representan los valores de la lógica booleana y están constituidos únicamente por las dos palabras reservadas true (verdadero) y false (falso). A diferencia de otros lenguajes, en Java no existe una equivalencia numérica directa entre el valor `0` o `1` y los valores booleanos.
- **Literales de carácter:** representan un único símbolo del juego de caracteres Unicode. Se delimitan mediante comillas simples e incluyen soporte para secuencias de escape.
- **Literales de cadena:** consisten en una secuencia de cero o más caracteres delimitados por comillas dobles. Aunque `String` no es un tipo de datos primitivo, Java proporciona este soporte sintáctico especial, convirtiendo automáticamente cualquier literal de este tipo en una instancia inmutable de la clase.
- **Literal nulo:** la palabra `null`, representa la ausencia explícita de referencia a un objeto dentro de una variable de tipo objeto o referencia, como `String`.

```
BooleanLiteral:
  (one of)
  true false

CharacterLiteral:
  ' SingleCharacter '
  ' EscapeSequence '
```

```
SingleCharacter:
  JavaLetterOrDigit but not ' or \

EscapeSecuence:
  \ t (horizontal tab HT, Unicode \u0009)
  \ n (linefeed LF, Unicode \u000a)
  \ r (carriage return CR, Unicode \u000d)
  \ LineTerminator (line continuation, no Unicode representation)
  \ “ (double quote ", Unicode \u0022)
  \ ’ (single quote ', Unicode \u0027)
  \ \ (backslash \, Unicode \u005c)

LineTerminator:
  the ASCII LF character, also known as "newline"
  the ASCII CR character, also known as "return"
  the ASCII CR character followed by the ASCII LF character

StringLiteral:
  " {StringCharacter} "

StringCharacter:
  JavaLetterOrDigit but not ' or \ EscapeSequence

NullLiteral:
  null
```

```
boolean activo = true;
char inicial = 'J';
char salto = '\n';
char unicodeA = '\u0041';
String saludo = "Bienvenido a Java";
Object objetoVacio = null;
```

Palabras reservadas

Como ya comentamos en el capítulo anterior, son aquellos *tokens* que tienen asignada una función específica dentro del lenguaje: los programadores tienen prohibido utilizarlas para cualquier otro propósito; por ello, no es posible nombrar una variable, un campo o una clase utilizando uno de estos términos. Al igual que en el lenguaje natural existen palabras con funciones gramaticales fijas, como las preposiciones, las palabras reservadas constituyen el vocabulario fundamental que el compilador reconoce para estructurar la lógica del programa.

Gran parte del proceso de aprendizaje de un lenguaje de programación consiste en familiarizarse con el significado y la utilidad de estos términos. En el caso de Java, el conjunto de estas palabras ha ido evolucionando con las diferentes versiones del lenguaje para adaptarse a nuevas necesidades. En la actualidad, el núcleo está compuesto por cincuenta y una palabras, a las que se suman dieciséis términos adicionales cuyo significado especial solo se activa dependiendo del contexto específico en el que se utilicen.

Es importante destacar que Java es un lenguaje extremadamente estricto con la forma en que se escriben estos elementos, siendo sensible a la diferencia entre mayúsculas y minúsculas. Todas las palabras reservadas se componen exclusivamente de letras minúsculas. Por esta razón, un término como **`int`** es reconocido como una palabra reservada para definir tipos enteros, mientras que `Int` o `INT` serían tratados como identificadores diferentes definidos por el usuario, aunque su uso se desaconseja para evitar ambigüedades en la lectura del código.

Dentro de este grupo, existen casos particulares como las palabras **`goto`** o **`const`**. Aunque figuran en la lista oficial y el programador no puede emplearlas como nombres de variables, actualmente no tienen ninguna función operativa dentro del lenguaje Java. Su reserva responde principalmente a razones históricas y al deseo de los diseñadores del lenguaje de evitar que programadores provenientes de otros entornos, como C o C++, intenten aplicar estructuras de programación que son incompatibles con la seguridad y la filosofía de Java.

A continuación, se presenta la relación de palabras reservadas en Java, estructurada conforme a la Especificación del Lenguaje Java[6]:

Palabras reservadas (*Keywords*)				
abstract	assert	boolean	break	byte
case	catch	char	class	const
continue	default	do	double	else
enum	extends	final	finally	float
for	goto	if	implements	import
instanceof	int	interface	long	native
new	package	private	protected	public
return	short	static	strictfp[7]	super
switch	synchronized	this	throw	throws
transient	try	void	volatile	while
_ (caracter de subrayado)				

Asimismo, existen los siguientes términos que funcionan como palabras reservadas dependiendo del contexto en el que se utilicen:

---

[6] La documentación y especificación oficial actualizada de Java puede consultarse en la plataforma Oracle Java Specification.

[7] La palabra strictfp fue introducida para restringir los cálculos de punto flotante a la norma IEEE 754; su uso es obsoleto.

Palabras reservadas dependientes del contexto (*Contextual Keywords*)

`exports`	`module`	`non-sealed`	`open`	`opens`
`permits`	`provides`	`record`	`requires`	`sealed`
`to`	`transitive`	`uses`	`var`	`when`
`with`	`yield`			

Finalmente, conviene recordar que el carácter de subrayado (`_`) por sí solo también se considera un elemento reservado para usos futuros[8], lo que prohíbe su empleo como un identificador de una sola letra en las declaraciones del programa.

Ahora que conocemos los literales y las palabras reservadas, vamos a modificar ligeramente una de las producciones sintácticas que veíamos en el tema anterior:

```
Identifier:
  IdentifierChars but not a Keyword or BooleanLiteral or NullLiteral
```

Es decir, un *identificador* es una secuencia ilimitada de letras y dígitos, comenzando por una letra, que no es una palabra reservada, un literal booleano o un literal nulo.

Expresiones y operadores

Una vez que el compilador ha concluido la fase de análisis léxico e identificado los *tokens* que componen nuestro código, el siguiente nivel de abstracción consiste en ensamblar estas piezas atómicas para formar unidades de significado superior. En la gramática de Java, estas construcciones se denominan **expresiones**.

Una expresión, como ya vimos en el tema 2, no es otra cosa que un conjunto organizado de identificadores, literales y operadores que, al combinarse siguiendo las reglas sintácticas del lenguaje, pueden ser evaluados para obtener un valor concreto.

El concepto de evaluación es central en esta etapa: evaluar una expresión implica que el ordenador realiza las operaciones especificadas para producir un nuevo valor. Es importante destacar que toda expresión en Java tiene asociado un tipo de dato inamovible, el cual corresponde al tipo del resultado final obtenido tras su evaluación. Así, el resultado de una expresión que sume dos números enteros será un valor de tipo entero, mientras que el de una que compare dos valores será de tipo booleano.

Podemos entender las expresiones como los bloques de construcción lógicos de un programa. No obstante, existe una distinción técnica crucial entre una expresión y una instrucción o sentencia. Mientras que la expresión representa un valor potencial o un cálculo, la **sentencia** representa una acción completa que el ordenador debe llevar a cabo. Por ejemplo, la expresión matemática $n + 1$ por sí sola no constituye una orden operativa para la máquina; necesita ser integrada en una estructura que le dé un propósito funcional, como una sentencia de asignación que guarde dicho resultado en una variable. De esta forma, las expresiones actúan como la «materia prima» con la que alimentamos las instrucciones de nuestro código para dirigir el flujo de control y resolver problemas complejos.

---

[8] A partir de Java 9 (JEP 213), el carácter de subrayado _ dejó de ser un identificador válido y pasó a ser un término reservado.

Operadores

Los **operadores** son los elementos funcionales que permiten llevar a cabo operaciones sobre un conjunto de datos u operandos, representados habitualmente por literales o identificadores. Los operadores permitidos en una operación dependen de los tipos de datos de los operandos y producen, tras su evaluación, un resultado que posee un tipo de dato determinado.

Operadores aritméticos

Son operadores que afectan a los números enteros y en punto flotante; devuelven un valor del mismo tipo que los operandos: si son enteros, por ejemplo, el resultado es entero. Los **operadores unarios** permiten mantener o cambiar el signo de una expresión numérica mediante el uso del más (`+`) y el menos (`-`). Junto a éstos, los **operadores binarios tradicionales**, que incluyen la suma (`+`), la resta (`-`), la multiplicación (`*`) y la división (`/`). Además, los lenguajes de programación suelen añadir la operación módulo o resto de la división (`%`).

Operador unario: Un operador que solo tiene un operando.
Operador binario: Un operador que tiene dos operandos.

En programación no es habitual usar el más unario y, cuando se trata de un literal, se asume que, si no hay signo, siempre es positivo.

Las operaciones de la suma, resta, producto y división, funcionan de la misma manera que te enseñaron cuando aprendiste a utilizarlas. La división en punto flotante, por ejemplo, genera un resultado en punto flotante:

```
7.2 / 2.0 produce 3.6
```

No obstante, es menos probable estar familiarizado con la división entera y el módulo, por lo que podemos estudiarlos un poco más en profundidad. Cuando dividimos dos enteros entre sí, obtemos de la división un cociente y un resto. Por ejemplo, la división de 6 entre 2 produce 3 como cociente y 0 como resto; pero la de 7 entre 3, produce también 3 como cociente pero 1 como resto. Y ese resto, 0 ó 1, es el resultado que obtenemos con la operación módulo:

```
6 / 2 produce 3     6 % 2 produce 0
7 / 2 produce 3     7 % 2 produce 1
```

Aunque existen lenguajes para los que el operador módulo solo es aplicable a la división entera, puede aplicarse en Java, y otros lenguajes modernos, con números en punto flotante. La forma de obtenerlo[9] en Java[10] consiste en obtener la división en punto flotante; multiplicar dicho valor sin decimales (sin redondeo) por el divisor; restar del dividendo el resultado de la multiplicación. Ejemplo:

```
Calculamos 7.8 % 3.0 en Java:
Dividimos: 7.8 / 3.0 produce 2.6
Multiplicamos: 3.0 * 2.0 produce 6.0
Restamos: 7.8 - 6.0 produce 1.8
```

```
Expression:
  Literal
  ArithmeticExpression
  AssignmentExpression

ArithmeticExpression:
  AdditiveExpression

AdditiveExpression:
  MultiplicativeExpression
  AdditiveExpression + MultiplicativeExpression
  AdditiveExpression - MultiplicativeExpression

MultiplicativeExpression:
  UnaryExpression
  MultiplicativeExpression * UnaryExpression
  MultiplicativeExpression \ UnaryExpression
  MultiplicativeExpression % UnaryExpression

UnaryExpression:
  + Expression
  - Expression
```

Operadores relacionales

Los operadores relacionales son binarios, por lo que tienen dos operandos. Se utilizan para comparar datos de tipo primitivo ya sean numéricos, caracteres o booleanos, estableciendo relaciones de orden o igualdad entre ellos. El resultado de evaluar una expresión con estos operadores es siempre un valor de tipo booleano[11]: true o false.

Java proporciona operadores para verificar:

- La igualdad (==)
- La desigualdad o diferencia (`!=`)

---

[9] El método estándar IEEE 754 puede obtenerse usando el método `Math.IEEEremainder`.

[10] Disponible en la especificación.

[11] Los booleanos corresponden a un tipo de datos lógico que solo pueden contener uno de dos valores: verdadero (`true`) y falso (`false`).

- Relaciones de magnitud: mayor que (>), menor que (<), mayor o igual que (>=) y menor o igual que (<).

```
Expression:
  Literal
  ArithmeticExpression
  EqualityExpression
  AssignmentExpression

EqualityExpression:
  RelationalExpression
  EqualityExpression == RelationalExpression
  EqualityExpression != RelationalExpression

RelationalExpression:
  AdditiveExpression
  RelationalExpression < AdditiveExpression
  RelationalExpression > AdditiveExpression
  RelationalExpression <= AdditiveExpression
  RelationalExpression >= AdditiveExpression
```

Cuidado con la asignación: Un error de lógica muy frecuente en etapas iniciales de aprendizaje consiste en confundir el operador de asignación (=) con el de igualdad (==), siendo el primero una instrucción de almacenamiento de un valor en memoria y el segundo una comparación.

Aunque en otros lenguajes resulte posible, en Java no existe relación de magnitud entre los valores booleanos, por lo que solo les son aplicables la igualdad y la desigualdad.

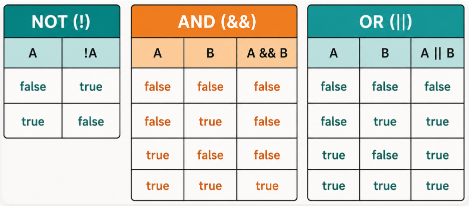
Operadores lógicos

Los operadores lógicos son binarios, por lo que tienen dos operandos, salvo la negación. Permiten combinar valores booleanos o resultados de expresiones relacionales para construir afirmaciones lógicas. Los operadores fundamentales son:

- La negación lógica (NOT) o NO-lógico (`!`).
- La conjunción (AND) o Y-lógico (&&).
- La disyunción (OR) u O-lógico (`||`).

Hay que entender desde el principio que `true` y `false`, como hemos visto al declarar *`BooleanLiteral`*, no son nombres de variable ni palabras reservadas: son dos constantes especiales pero, en la práctica, se comportan como dos palabras reservadas.

El operador NO-lógico precede a cualquier expresión lógica (booleana) y nos proporciona el valor opuesto al de la expresión. Por ejemplo, si `miCaracter = 'B'` es verdadero, `!(miCaracter = 'B')` es falso. Proporciona un método simple de cambiar el valor de un aserto.



NOT (!)

A	!A
false	true
true	false

AND (&&)

A	B	A && B
false	false	false
false	true	false
true	false	false
true	true	true

OR (||)

A	B	A || B
false	false	false
false	true	true
true	false	true
true	true	true

Por ejemplo, si `!(años > 50)`, es equivalente escribir: `años <= 50`.

La operación Y-lógico requiere que ambos operandos sean verdaderos para que el resultado de la expresión sea verdadero; si uno cualquiera de ellos es falso, el valor de la expresión es falso.

La operación O-lógico proporciona un resultado verdadero si cualquiera de sus operandos, que pueden serlo los dos, es verdadero. Solo cuando ambos operandos son falsos el resultado de la operación es también falso.

```
Expression:
  Literal
  ArithmeticExpression
  EqualityExpression
  ConditionalExpression
  AssignmentExpression

ConditionalExpression:
  ConditionalOrExpression

ConditionalOrExpression:
  ConditionalAndExpression
  ConditionalOrExpression || ConditionalAndExpression

ConditionalAndExpression:
  Expression
  ConditionalAndExpression && Expression
```

Java implementa para estos dos últimos una semántica de **evaluación en cortocircuito**, lo que significa que el ordenador detiene el proceso de evaluación tan pronto como el resultado final de la expresión booleana es inequívoco y que explicaremos en breve.

Operador de asignación

Aunque en muchos lenguajes la asignación se considera una sentencia o instrucción, en Java, el símbolo igual (=) es un operador, el operador de asignación, cuya misión es proporcionar valor a una variable, el de una expresión a su derecha, como ya vimos en el tema anterior.

Evaluación en cortocircuito

Al procesar expresiones lógicas, la mayoría de los lenguajes de programación no garantizan el orden en que se evaluarán dichas expresiones: esto quiere decir que si una operación lógica incluye operaciones relacionales, por ejemplo, no sabemos el orden en que se evaluarán dichas operaciones. Además, en general, la operación lógica no se evaluará hasta que todos los operandos de la misma obtengan un valor booleano.

Java aplica una técnica de optimización denominada **evaluación en cortocircuito** o evaluación condicional. Bajo esta semántica, el ordenador no evalúa todos los componentes de una expresión lógica, sino que procesa los operandos de izquierda a derecha y detiene el procedimiento de evaluación tan pronto como el valor booleano final de la expresión completa es inequívoco.

Evaluación (lógica) en cortocircuito: Evaluación de una expresión lógica de izquierda a derecha, deteniéndose la evaluación tan pronto como se pueda determinar el valor booleano final.

Para comprender cómo el ordenador puede conocer el resultado sin examinar la expresión entera, debemos analizar el comportamiento de los operadores fundamentales:

- **Conjunción lógica (&&):** Una operación Y-lógico solo devuelve true si *ambos* operandos son verdaderos. Por tanto, si al evaluar el primer operando el resultado es false, es imposible que la expresión completa sea verdadera, independientemente del valor que tenga el segundo operando. En este caso, Java «hace un cortocircuito», detiene la evaluación y produce un resultado final de false.
- **Disyunción lógica (||):** Una operación O-lógico devuelve true si *al menos uno* de sus operandos es verdadero. Siguiendo la lógica anterior, si el primer operando evaluado resulta ser true, el resultado final de la expresión será necesariamente true sin importar el valor del segundo. En consecuencia, el ordenador no pierde tiempo procesando la segunda subexpresión.

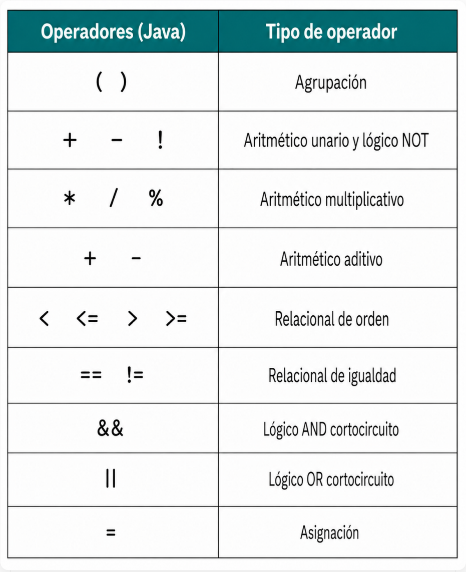
Esta característica no es solo una cuestión de eficiencia técnica para ahorrar tiempo de ejecución; tiene implicaciones críticas en la robustez y seguridad del código. La evaluación en cortocircuito permite al programador escribir expresiones donde el primer operando actúa como una salvaguarda del segundo.

Precedencia y asociatividad

El orden en que el ordenador lleva a cabo las operaciones dentro de una sentencia no es un proceso arbitrario, sino que está estrictamente regulado por las leyes de la gramática. Al construir expresiones complejas donde conviven distintos tipos de componentes, es fundamental determinar con exactitud qué operación debe ejecutarse en primer lugar para que el resultado sea predecible y correcto. Para resolver este problema, el lenguaje emplea un conjunto de directrices denominadas **reglas de precedencia y asociatividad**.

Reglas de precedencia

La precedencia determina la jerarquía de los operadores en una expresión. De forma análoga a cómo en el álgebra convencional sabemos que una multiplicación debe realizarse antes que una suma, Java asigna un nivel de prioridad a cada operador. Los operadores con mayor precedencia se evalúan antes que aquellos situados en niveles inferiores.



Operadores (Java)	Tipo de operador
( )	Agrupación
+ - !	Aritmético unario y lógico NOT
* / %	Aritmético multiplicativo
+ -	Aritmético aditivo
< <= > >=	Relacional de orden
== !=	Relacional de igualdad
&&	Lógico AND cortocircuito
||	Lógico OR cortocircuito
=	Asignación

En la cima de esta jerarquía se sitúan siempre los paréntesis `()`, que no son operadores en sí mismos pero funcionan como una herramienta soberana para que el programador pueda forzar el orden de evaluación deseado, sobrescribiendo cualquier regla predefinida.

En el extremo opuesto, el operador de asignación posee la precedencia más baja de todo el lenguaje, lo que garantiza que cualquier cálculo situado a la derecha del símbolo igual sea completado totalmente antes de que el valor resultante se almacene en la variable.

Asociatividad

Cuando una expresión contiene varios operadores que poseen el *mismo* nivel de precedencia, Java aplica las reglas de asociatividad para decidir el orden de ejecución. La asociatividad indica la dirección en la que el ordenador «agrupa» los operandos con sus respectivos operadores.

En la gran mayoría de los casos, los operadores binarios de Java presentan una asociatividad de **izquierda a derecha**. Esto significa que, ante una igualdad de rango, las operaciones se resuelven en el mismo orden físico en que aparecen escritas en el código. Por ejemplo, en una expresión que combine multiplicaciones y divisiones, el ordenador ejecutará primero la que se encuentre más a la izquierda.

Sin embargo, existen excepciones técnicas notables: los operadores unarios (como el cambio de signo o la negación lógica) y, muy especialmente, el operador de asignación, son **asociativos por la derecha**. Esta asociatividad a la derecha es lo que permite realizar asignaciones múltiples, donde el valor se propaga desde el literal situado al final de la línea hacia todas las variables situadas a su izquierda.

Orden de evaluación de operandos

Es vital no confundir la precedencia de los operadores con el orden de evaluación de los operandos. Java establece un requisito adicional de seguridad y robustez: **los operandos de un operador binario siempre se evalúan de izquierda a derecha**. Esto implica que, incluso si el operador situado a la derecha tiene mayor precedencia, el ordenador primero obtendrá los valores de la parte izquierda de la expresión. Esta característica es crítica para evitar efectos laterales[12] inesperados al realizar trazas de ejecución en algoritmos complejos.

Aunque las reglas de precedencia permiten escribir expresiones muy compactas, se recomienda el uso de paréntesis para clarificar la intención del código[13]. El empleo de paréntesis no solo ayuda a evitar errores de lógica difíciles de detectar, sino que facilita la lectura, asegurando que el flujo de control y el procesamiento de los datos sigan exactamente el diseño algorítmico previsto.

Conversión de tipos

En el transcurso de este texto, hemos subrayado que Java es un lenguaje fuertemente tipado, lo que implica una vigilancia estricta sobre la naturaleza de los tipos de datos que manejamos. Sin embargo, en la práctica de la programación, es habitual encontrarnos con situaciones en las que necesitamos combinar operandos de distinta naturaleza dentro de una misma expresión o asignación. Por ejemplo, podríamos querer sumar un valor entero con uno real o asignar el resultado de un cálculo preciso a una variable con una capacidad de representación diferente.

Dado que el ordenador almacena internamente los números enteros y los de punto flotante de maneras totalmente distintas —utilizando patrones de bits que no guardan parecido entre sí—, el sistema no puede operar con ellos de forma directa sin realizar un ajuste previo.

Este proceso de transformación se denomina técnicamente **conversión de tipos**. La conversión es el mecanismo que garantiza la compatibilidad entre los elementos de una operación, permitiendo que el compilador traduzca nuestras intenciones lógicas a instrucciones que la máquina virtual pueda procesar sin ambigüedades. Dependiendo de cómo se inicie este proceso, las conversiones pueden ser **implícitas**, cuando el compilador las realiza de forma automática al detectar una mezcla de tipos en una expresión, o **explícitas**, cuando nosotros, como programadores, debemos intervenir deliberadamente mediante una operación específica conocida como *casting*.

Conversión de tipos (*Type Conversion*): Proceso mediante el cual un valor de un tipo de dato determinado se transforma en otro equivalente de un tipo diferente.

Un aspecto crítico que debemos considerar al tratar con estas transformaciones es la seguridad de la información. No todas las conversiones son iguales desde el punto de vista del rigor de los datos; mientras que algunas son seguras porque el tipo de destino es capaz de representar perfectamente el valor original, otras pueden implicar una pérdida de precisión o el truncamiento de valores decimales. Por ello, el uso de conversiones explícitas no es solo un requisito gramatical en ciertos contextos, sino también una práctica de calidad que deja claro que la mezcla de tipos es intencionada y no fruto de un descuido. Dominar la mecánica de estas conversiones es esencial para asegurar que los datos se procesen con exactitud.

---

[12] Cualquier función o línea de código que modifica algo fuera de su entorno local. Observad que es *lateral*, no *colateral*; debería denominarse efecto secundario pero surge de su traducción literal del inglés, *side effect*, siendo el término más extendido.

[13] Es evidente que esta afirmación carece de sentido si estás haciendo un examen en el que debes demostrar conocer cuál es el orden de precedencia y asociatividad de los operadores.

Reglas de promoción

Las reglas de promoción constituyen la base de las **conversiones implícitas** o automáticas. Este fenómeno ocurre sin la intervención directa del programador cuando el compilador detecta que en una operación aritmética o lógica conviven operandos de distinta precisión o capacidad de representación. En estos escenarios, para poder ejecutar la instrucción, el sistema «promociona» el valor de menor rango al tipo de datos más amplio presente en la expresión, garantizando que no se produzca una pérdida de información durante el cálculo.

Técnicamente, estas transformaciones se denominan **conversiones de ampliación** (*widening conversions*), ya que el tipo de destino posee una capacidad igual o superior al de origen, lo que las convierte en operaciones intrínsecamente seguras. En el caso de Java se sigue una jerarquía estricta para realizar estos ajustes automáticos, basada en el rango numérico y en la distinción entre tipos enteros y reales:

- **Prioridad de la precisión real:** Si al menos uno de los operandos en una expresión es de tipo `double`, el otro operando se convierte automáticamente a `double` antes de realizar la operación. En caso de que no haya un `double` pero sí un componente de tipo `float`, el resto de los elementos se promocionan a `float`.
- **Jerarquía de tipos enteros:** Si no existen tipos reales involucrados, pero uno de los operandos es de tipo `long`, el sistema promociona los demás operandos de la expresión a `long`[14].
- **Promoción obligatoria a entero:** Un aspecto singular de Java es el tratamiento de los tipos de menor capacidad. En cualquier cálculo donde intervengan variables de tipo `byte` o `short`, el compilador las promociona automáticamente a `int` antes de efectuar la operación, incluso si ambos operandos son del mismo tipo.
- **Tratamiento de caracteres:** De igual forma, el tipo `char` recibe un trato especial; dado que Java almacena los caracteres como valores numéricos sin signo (basados en el estándar Unicode), estos pueden participar en expresiones aritméticas, en cuyo caso se promocionan automáticamente a su equivalente de tipo `int`.

Este mecanismo asegura que los resultados intermedios de una expresión se almacenen en variables temporales con el tipo de mayor capacidad que intervenga en la operación, permitiendo que el software maneje la complejidad de los datos de forma fluida. No obstante, aunque la promoción automática facilita el desarrollo al evitar farragosas conversiones manuales, el programador debe conocer estas reglas para prever el tipo de dato resultante y asegurar que el procesamiento de la información sea coherente con el diseño previsto.

---

[14] Resulta interesante notar que un long (entero de 64 bits) se promociona automáticamente a float (real de 32 bits) si coinciden en una expresión. Aunque el float ocupa menos memoria, su notación científica le otorga una capacidad de representación de magnitudes exponencialmente mayor, considerándose por tanto un tipo más «amplio», aunque se pueda perder precisión en los dígitos menos significativos. También se puede producir pérdida de precisión en las conversiones de int a float y de long a double.

Conversión explícita

A diferencia de las reglas de promoción, que actúan de forma automática, existen situaciones en las que el sistema no puede —o no debe— realizar el ajuste de tipos por su cuenta. Esto sucede principalmente cuando intentamos realizar una *narrowing conversion*, en la que queremos asignar un valor de un tipo con mayor capacidad de representación a una variable de un tipo menor o cuando la transformación implica un riesgo intrínseco de pérdida de información. En estos casos, Java requiere una orden directa del programador mediante una operación denominada **conversión explícita** o, más habitualmente, ***casting***.

Casting: Operación explícita indicada por el programador para forzar la conversión de un tipo de dato a otro.

Widening conversions: Conversión de ampliación de un tipo de datos a otro de mayor capacidad o precisión.

Narrowing conversion: Conversión de reducción de un tipo de datos a otro de menor capacidad o precisión , asumiendo el riesgo de pérdida de información.

Para indicar al compilador nuestra intención de realizar un *cast*, debemos utilizar una sintaxis específica, colocando el tipo de destino entre paréntesis justo antes de la expresión que deseamos transformar.

```
int convertido = (int)3.1416;
```

A efectos prácticos, aunque no lo es, podéis considerarlo un operador cuya precedencia se situaría entre los operadores unarios y los aritméticos multiplicativos.

```
UnaryExpression:
  + Expression
  - Expression
  UnaryExpressionNotPlusMinus

UnaryExpressionNotPlusMinus:
  CastExpression

CastExpression:
  ( PrimitiveType ) UnaryExpression

PrimitiveType:
  NumericType
  boolean

NumericType:
  IntegralType
  FloatingPointType
```

```
IntegralType:
  (one of)
  byte short int long char

FloatingPointType:
  (one of)
  float double
```

Al realizar un *cast*, el programador asume la responsabilidad total de la operación, informando al compilador de que es consciente del riesgo y de que la mezcla de tipos es intencionada. El uso de la conversión explícita conlleva consecuencias críticas en la integridad de los datos que debemos conocer para evitar errores lógicos:

- **Truncamiento de decimales:** Cuando se convierte un número real (como un `double` o `float`) a un tipo entero (`int`, `long`, etc.), Java no realiza un redondeo matemático al valor más cercano; en su lugar, trunca: elimina la parte fraccionaria. Por ejemplo, realizar un *cast* a entero del valor 3.99 producirá como resultado el número entero 3.
- **Pérdida de bits significativos:** Si intentamos forzar un número entero grande en un tipo que utiliza menos bits (como pasar de `int` a `byte`), el ordenador descartará los bits más significativos para que el valor encaje en el espacio de destino[15]. Resulta imprescindible validar el rango del número a convertir.
- **Restricciones de tipo:** No todas las conversiones son posibles. Por ejemplo, en Java es estrictamente imposible convertir tipos numéricos a booleanos, ni siquiera mediante conversión explícita.

Clases envolventes

Para completar nuestra comprensión sobre la transformación de datos, es imprescindible abordar el papel de las **clases envolventes**, conocidas como *wrappers*.

Java mantiene una distinción fundamental entre los tipos de datos primitivos y los referenciados. Sin embargo, existen numerosos escenarios en los que necesitamos tratar un valor simple como si fuera un objeto, ya sea para utilizar ciertas funcionalidades de la biblioteca estándar o para facilitar procesos de conversión complejos que los tipos básicos no pueden realizar por sí mismos.

Las clases envolventes son, en esencia, clases diseñadas para actuar como un complemento de los tipos primitivos, empaquetando un valor elemental dentro de una estructura de objeto. Java proporciona una de estas clases para cada uno de los tipos fundamentales: `Byte`, `Short`, `Integer`, `Long`, `Float`, `Double`, `Character` y `Boolean`. Observamos que, siguiendo las reglas de nomenclatura del lenguaje, sus nombres comienzan siempre con una letra mayúscula. El nombre de la clase envolvente coincide con el del tipo primitivo, excepción hecha de `int` y `char`, que se corresponden, respectivamente, con `Integer` y `Character`.

---

[15] Internamente, al pasar a un tipo entero más pequeño, elimina los bits de la izquierda. En la representación binaria en complemento a dos, esto puede alterar el bit de signo: por ejemplo, la conversión de 1234 a byte produce el número -46.

Java convierte automáticamente de un tipo primitivo a un objeto de la clase envolvente equivalente, **boxing**, y viceversa, **unboxing**. Debemos tener en cuenta que solo es posible la conversión entre tipos equivalentes, es decir, que no podemos, por ejemplo, convertir un `int` en `Byte`. Y aunque podamos hacer el *cast* entre tipos primitivos, ello tampoco es posible entre objetos de las clases envolventes.

Estas clases, además, nos proporcionan constantes y métodos asociados a los tipos primitivos que representan como: `Integer.MIN_VALUE` e `Integer.MAX_VALUE`, los valores enteros menor y mayor representables en Java.

En el ámbito de la conversión de tipos, su utilidad reside en la capacidad para transformar información textual en valores operativos para el ordenador. Dado que muchas operaciones de entrada de datos devuelven los resultados en forma de un objeto `String`, el programador debe recurrir a métodos específicos. Métodos como `Integer.parseInt` o `Double.parseDouble` permiten analizar el contenido de una cadena y obtener su valor numérico. Es vital validar estas operaciones , pues si el texto analizado no representa un número válido (por ejemplo, intentar convertir “Hola” a un entero), el sistema interrumpirá el flujo de control una excepción de tipo `NumberFormatException`.

Uso de la biblioteca estándar

Hasta ahora hemos utilizado algunas operaciones que forman parte directamente de las posibilidades del lenguaje Java. Podemos realizar operaciones aritméticas, comparar valores, construir expresiones lógicas y almacenar sus resultados en variables o realizar llamadas a métodos.

Sin embargo, un lenguaje de programación no tendría mucho sentido si obligara al programador a desarrollar desde cero todas las operaciones que puede necesitar. Pensemos, por ejemplo, en una aplicación que tenga que calcular una raíz cuadrada. Podríamos intentar desarrollar nosotros mismos un algoritmo para calcularla, del mismo modo que podríamos escribir nuestros propios algoritmos para obtener potencias, calcular valores absolutos o trabajar con funciones trigonométricas. Pero hacerlo cada vez que necesitáramos una de estas operaciones sería innecesario y, además, nos obligaría a resolver problemas que ya han sido resueltos muchas veces.

Una de las ventajas de utilizar un lenguaje de programación orientado a objetos, como Java, es que no es necesario partir de cero. Cualquier programa que desarrollemos, por sencillo que parezca, se apoya y se estructura sobre una inmensa base de código preexistente. Esta infraestructura es lo que se conoce como **biblioteca estándar** (en Java, de manera formal, **la API de Java** o ***Application Programming Interface***).

Como vimos en el tema anterior, una biblioteca no es algo diferente al propio lenguaje que nos proporciona nuevas instrucciones mágicas: es una colección de clases y métodos predefinidos que realizan tareas específicas de uso común. Lo que hace es poner a nuestra disposición código escrito previamente y verificado, que podemos utilizar desde nuestros programas. De esta manera, podemos construir soluciones cada vez más complejas combinando las capacidades del lenguaje con las que proporcionan sus bibliotecas, lo que no solo acelera el desarrollo de software, sino que también garantiza la robustez, eficiencia y confiabilidad del código resultante.

Biblioteca estándar: Conjunto de clases, interfaces y métodos preescritos que acompañan a un lenguaje de programación, ofreciendo a los desarrolladores herramientas listas para usar en la resolución de problemas comunes.

La biblioteca estándar de Java es especialmente extensa. Como ya comentamos en el tema 1, para evitar que el programador se sienta abrumado por el volumen gigantesco de recursos que componen esta biblioteca, la plataforma organiza sus clases en agrupaciones lógicas denominadas **paquetes** (*packages*), que veremos a lo largo del curso.

Existe, sin embargo, un paquete elemental llamado `java.lang`; este agrupa las clases de uso más frecuente e indispensable para el núcleo de la programación (como, por ejemplo las clases `String` y `System`). En este momento nos interesa una de las clases más sencillas de utilizar y, al mismo tiempo, una de las que más frecuentemente encontraremos durante nuestros primeros programas: la clase `Math`.

La clase Math

La clase `Math` es una de las herramientas de utilidad más importantes de la biblioteca estándar de Java. Proporciona operaciones matemáticas que podemos utilizar directamente en nuestros programas. Entre ellas encontramos funciones para calcular valores absolutos, potencias, raíces cuadradas, valores máximo y mínimo, redondeos y otras muchas operaciones.

Su utilización nos permitirá resolver problemas matemáticos sin tener que desarrollar previamente el algoritmo correspondiente. Por ejemplo, si necesitamos obtener la raíz cuadrada de un número, no tenemos que escribir un algoritmo que la calcule. Podemos utilizar directamente el método proporcionado por la clase `Math`:

```
double numero = 25.0;
double raiz = Math.sqrt(numero);
System.out.println(raiz);
```

El resultado que obtendremos será:

```
5.0
```

La expresión `Math.sqrt(numero)` es una llamada a un método, como lo son `print` o `println`. La diferencia que encontramos ahora es que, en lugar de hacerlo desde un objeto, invocamos el método desde la misma clase `Math` y lo identificamos escribiendo primero el nombre de la clase, seguido de un punto y del nombre del método.

Podemos interpretar la expresión anterior de la siguiente forma:

```
Math     .     sqrt     ( numero )
 ↑               ↑          ↑
clase          método    argumento
```

El punto (`.`) permite acceder a un elemento proporcionado por una clase u objeto. En este caso, escribimos `Math.sqrt` porque queremos utilizar el método `sqrt` de la clase `Math`[16].

Observemos que `sqrt` necesita un argumento: el número del que buscamos la raíz cuadrada. El método realiza la operación y devuelve un resultado, que podemos utilizar como cualquier otro valor obtenido mediante una expresión.

Por ejemplo:

```
double resultado = 2 * Math.sqrt(9);
```

En esta ocasión, primero se obtiene el valor proporcionado por `Math.sqrt(9)`, que es `3.0`, y posteriormente se realiza el producto:

Por tanto, la variable resultado terminará conteniendo el valor `6.0`.

La clase `Math` proporciona numerosos métodos. No es necesario aprenderlos todos de memoria. Lo más importante es aprender a reconocer qué operación necesitamos y saber consultar la documentación de la biblioteca para encontrar el método apropiado, conocer sus parámetros y saber qué tipo de valor devuelve.

A continuación, vemos algunos de los elementos que pueden ser más interesantes para nosotros en este momento:

- **Math.PI**: Representa el valor de la constante π (aproximadamente 3.141592653589793).
- **Math.abs(x)**: Calcula el valor absoluto de su argumento, eliminando el signo negativo si lo tuviera. Soporta tanto número enteros como reales.
- **Math.pow(base, exponente):** Eleva el primer argumento (la base) a la potencia especificada por el segundo argumento (el exponente), gestionando automáticamente casos especiales como potencias fraccionarias o bases negativas.Tanto los argumentos como el resultado son tratados como **`double`**.
- **Math.round(x)**: Redondea un número decimal al entero más cercano. Si recibe un **`double`** devuelve un **`long`**; si recibe un **`float`** devuelve un **`int`**.

---

[16] Explicaremos la diferencia que existe entre invocar un método de un objeto o una clase en el tema 4.

- **Math.floor(x)**: Realiza un redondeo hacia abajo, devolviendo el mayor valor entero (representado como un `double`) que sea menor o igual al argumento decimal.

La utilización de una biblioteca no significa que podamos olvidar la naturaleza de la operación matemática que estamos realizando. La biblioteca nos proporciona los mecanismos para efectuarla, pero sigue siendo responsabilidad del programador comprender qué datos necesita cada método y cómo interpretar el resultado.

Generación de números aleatorios

La vida real presenta en muchas ocasiones situaciones que se producen por azar. Cuando nosotros queremos modelar en nuestros programas estas situaciones, recurrimos a generadores de números aleatorios. En realidad, en nuestros programas, lo que usamos no son números aleatorios -aunque los denominemos así- sino pseudoaleatorios[17]: se generan a través de fórmulas matemáticas que usan algoritmos deterministas y periódicos, que cumplen pruebas estadísticas de aleatorioedad.

El uso de los números aleatorios será algo que ocurra con frecuencia en nuestros ejercicios, por lo que utilizaremos un método que nos proporciona la clase `Math`: `random`.

Una llamada:

```
Math.random();
```

devuelve un número de tipo `double` mayor o igual que 0.0 y menor que 1.0.

Es importante observar que no podemos saber de antemano cuál será el valor concreto obtenido. Esto permite utilizar `Math.random` en programas que necesitan introducir cierta variabilidad en su comportamiento.

Podemos combinar este resultado con operaciones aritméticas para obtener valores en otros intervalos: multiplicando el resultado por el valor del límite y realizando una conversión explícita (*casting*) a tipo entero para truncar los decimales. Por ejemplo, la asignación:

```
int numero = (int) (Math.random() * 10);
```

da como resultado que la variable `numero` tome un valor entero entre 0 y 9. El uso de los paréntesis es importante: sin ellos, la conversión se aplicaría únicamente al valor devuelto directamente por `Math.random()`, antes de realizar la multiplicación, lo que demuestra la importancia de comprender la precedencia de los operadores y utilizar los paréntesis cuando queremos expresar de forma clara el orden en que deben realizarse las operaciones.

---

[17] Existen los denominados TRNG (*True Random Number Generators*) que extraen valores aleatorios de fenómenos físicos impredecibles.

En general, para obtener un número en un rango [A, B] realizaremos un escalado:

```
int numeroAleatorio = (int) (Math.random() * (B - A + 1)) + A;
```

Explicamos con un poco más de detalle lo que hemos hecho. Sabemos que `random` generará un número entre 0.0 y 1.0, excluido. Queremos obtener tantos números como hay entre A y B; si, por ejemplo, el rango fuera entre 10 y 20, querríamos obtener once números: 10, 11, 12, 13, 14, 15, 16, 17, 18, 19 y 20; es decir, B - A + 1, porque queremos que B esté incluido. Por el ejemplo anterior, hemos visto que al multiplicar el resultado de random por un número, y convertirlo a entero, nos proporciona una cantidad de números igual al multiplicando: si seguimos con el ejemplo, al multiplicarlo por 11 nos proporciona los números del 0 al 10. Ahora solo tenemos que «desplazar» el cero hasta A, sumándolo al resultado de la operación anterior.

Importación de Math

Como vemos, cada vez que invocamos un método de la biblioteca debemos escribir el nombre de la misma. En el desarrollo de programas que hagan mucho uso de sus métodos, repetir constantemente el prefijo `Math` puede hacer que escribir el código sea tedioso pero, aún peor, que sea incómodo de leer. Como ya vimos, **`import`** nos permite usar los objetos de una biblioteca pero, en el caso de `Math`, queremos importar sus métodos, cosa que hacemos añadiendo la palabra **`static`**.

```
ImportDeclaration:
  SingleTypeImportDeclaration
  SingleStaticImportDeclaration

SingleStaticTypeImportDeclaration:
  import static PackageOrTypeName . Identifier
  import static PackageOrTypeName . *
```

Así, podemos traer de forma directa al ámbito de nuestro archivo fuente una función individual de la clase `Math` o importar todas sus constantes y métodos de manera colectiva utilizando el comodín (`*`):

```
// Importación de funciones individuales
import static java.lang.Math.sqrt;
import static java.lang.Math.pow;

// Importación estática de todos los miembros de la clase Math
import static java.lang.Math.*;
```

Una vez declarada la importación al inicio de nuestro fichero, ya no es necesario calificar los métodos con el nombre de la clase: podemos escribir ecuaciones complejas con una sintaxis limpia y natural (como `sqrt(pow(ladoA, 2) + pow(ladoB, 2))`), que es más claro, mejorando la legibilidad.

La biblioteca como herramienta de programación

La clase `Math` constituye un ejemplo sencillo de una idea mucho más general. Cuando programamos, no siempre debemos construir desde cero todas las operaciones que necesita nuestro problema.

El lenguaje Java, como la mayoría de los lenguajes modernos, proporciona una gran cantidad de funcionalidades ya implementadas que podemos incorporar. Algunas pertenecen directamente al lenguaje y otras se encuentran organizadas en clases de su biblioteca estándar: su uso forma parte de la actividad habitual de un programador.

Por este motivo, cuando aprendamos una nueva clase de la biblioteca estándar no intentaremos memorizar todos sus métodos. Aprenderemos qué problema resuelve, conoceremos algunas de las operaciones que proporciona pero, sobre todo, aprenderemos a consultar su documentación para utilizar correctamente los elementos que necesitemos.

Instrucciones

Hasta ahora hemos trabajado principalmente con **expresiones**. Hemos visto que una expresión combina valores, variables y operadores y que, después de ser evaluada, produce un resultado. También hemos comprobado que una expresión puede formar parte de una operación más compleja o utilizarse para obtener el valor que almacenaremos en una variable. Pero un programa no está formado únicamente por expresiones. Mientras una expresión representa un cálculo o un valor, un programa necesita además indicar **qué debe hacer el ordenador con esos valores y en qué orden debe hacerlo**. Para ello utilizamos las **instrucciones** o **sentencias**, que constituyen verdaderas acciones con sentido completo.

Desde una perspectiva de alto nivel, una instrucción representa la unidad de ejecución más pequeña de un lenguaje de programación. En esencia, equivale a una orden precisa que le transmitimos a la máquina: cuando escribimos un programa, construimos una sucesión de instrucciones que el ordenador irá ejecutando de acuerdo con las reglas del lenguaje. Así, un programa puede definirse conceptualmente como una secuencia estructurada y ordenada de instrucciones orientadas a alcanzar un objetivo concreto. Algunas instrucciones permiten almacenar un resultado, otras tomar una decisión, repetir una operación o llamar a un método y, finalmente, existen directivas estructurales que gobiernan el orden dinámico en que se ejecutan las demás órdenes bajo determinadas condiciones (sentencias de **flujo de control**). Todas ellas tienen en común que forman parte de la solución concreta del algoritmo que estamos escribiendo en Java.

Por ejemplo, en el siguiente fragmento:

```
int edad = 20;
int añoNacimiento = 2026 - edad;
System.out.println(añoNacimiento);
```

tenemos tres instrucciones. La primera declara una variable y le proporciona un valor; la segunda realiza un cálculo y almacena su resultado; la tercera utiliza un método para mostrar información.

Aunque utilicemos los términos *instrucción* y *sentencia* con un significado prácticamente equivalente, en realidad, instrucción es una orden que indica a la máquina que realice una acción; sin embargo, **sentencia** (*statement*) es la unidad gramatical de código, con una estructura formal estricta, que ejecuta una acción completa.

Sentencias

Una aplicación está formada por sentencias que, en ausencia de una estructura que modifique el flujo de ejecución, se ejecutan de arriba abajo, en el orden en que aparecen. Ya conocemos algunas sentencias sencillas. Una declaración de variable, una asignación o una llamada a un método pueden constituir una sentencia:

```
int edad = 20;
edad = edad + 1;
System.out.println(edad);
```

La mayoría de las sentencias individuales, como las del ejemplo, se denominan sentencias simples y se caracterizan de forma estricta por finalizar con el carácter **punto y coma** (`;`). El punto y coma indica al compilador que hemos terminado una sentencia: no actúa como un mero separador de instrucciones sino como un **token terminador obligatorio**. No debemos confundirlo con los operadores que hemos estudiado anteriormente. El punto y coma no realiza ninguna operación sobre los datos; simplemente marca el final de determinadas construcciones sintácticas. La sintaxis del lenguaje es lo suficientemente flexible como para permitir la inclusión de múltiples sentencias consecutivas dentro de una única línea física de código (como en la instrucción `i = 0; j = 5; x = i + j;`); pero se recomienda escribir cada sentencia en una única línea para favorecer la claridad y la legibilidad.

Lo anterior puede llevarnos a asumir que cualquier expresión válida se transforma automáticamente en una sentencia por el simple hecho de añadirle un punto y coma al final. Esto no es así. Para que una expresión finalizada en punto y coma sea una sentencia legítima, esta debe albergar un significado operativo claro: debe desencadenar una acción concreta o un cambio de estado. Si intentamos añadir un punto y coma a una expresión como un literal aislado (`3.0;`), un identificador de variable (`numero;`) o una cadena de texto (`"hola;"`), el compilador rechazará el código emitiendo un error de sintaxis.

Java permite utilizar diferentes tipos de sentencias. Algunas ya las hemos utilizado, como las declaraciones, las asignaciones y las llamadas a métodos, según las plantillas sintácticas que vimos en el tema 2. En general, podemos distinguir, entre otras, las siguientes clases de sentencias:

- **Expresiones de asignación:** Aquellas cuyo operador principal es el de asignación (=). Almacenar o actualizar un valor en una variable representa una de las acciones más elementales que le podemos encomendar a una computadora. Resulta vital recordar que no se debe confundir el operador de asignación (=) con el de comparación de igualdad (==).
- **Invocaciones a métodos:** Llamar a un método de la biblioteca estándar es una acción en sí misma, independientemente de que el método devuelva o no un resultado. Ejemplos de ello son sentencias tan cotidianas como `System.out.println("Hola");` o como `Math.abs(3.0);`.
- **Sentencias de declaración:** Aquellas cuya finalidad es reservar un espacio en memoria para almacenar información. Al declarar una variable (como `int x;`), le ordenamos al programa que compruebe el tipo de dato para determinar su tamaño exacto en memoria, busque una zona de almacenamiento físico que esté libre en ese instante y asocie de manera permanente dicho espacio con el identificador que hemos elegido.
- **Sentencias de flujo de control:** Su misión fundamental es gobernar el orden preciso y las condiciones específicas bajo las cuales deben ejecutarse las acciones individuales del código. Son las encargadas de dictar el comportamiento dinámico del programa ante la toma de decisiones (selección) y la repetición (bucles).

Sentencia compuesta

Una sentencia simple representa una única acción del programa y, como hemos visto, normalmente termina con un punto y coma. Sin embargo, en muchas ocasiones, necesitamos ejecutar no una única sentencia, sino un conjunto de ellas, como si fueran una sola. En la mayoría de los lenguajes podemos agrupar sentencias delimitándo dicha agrupación mediante palabras reservadas, símbolos especiales o mediante indentación[18]. A estas de agrupaciones de sentencias se las demonima **sentencias compuestas**.

Sentencia compuesta: Conjunto de sentencias agrupadas dentro de un bloque de código, que puede utilizarse sintácticamente como una única sentencia.

En el el caso de Java, en cualquier lugar físico o lógico del código donde la sintaxis formal del lenguaje permita ubicar una única sentencia individual, el programador tiene la facultad de colocar una sentencia compuesta. Es preciso delimitarla mediante el uso de llaves de apertura y cierre (`{ }`). La agrupación se denomina **bloque** de código.

```
{
    sentencia1;
```

---

[18] Indentación es un anglicismo (indentation) que significa sangría, consistente en desplazar sentencias a la derecha añadiendo espacios o tabulación.

```
    sentencia2;
    sentencia3;
}
```

Ya vimos en la unidad anterior la plantilla sintáctica *`Block`*, que se utiliza para definir el bloque de código. La importancia de la sentencia compuesta se apreciará especialmente al utilizar sentencias de control.

Bloques y ámbito

El cometido de una sentencia compuesta es agrupar lógicamente varias instrucciones para que actúen como una única unidad de acción. Debido a que puede ser ubicada en cualquier lugar del código donde las reglas del lenguaje lo permitan, la indentación o sangrado del código en su interior no es un mero capricho estético: es una práctica metodológica fundamental que permite, al lector, identificar de un solo vistazo el inicio y el fin de cada sentencia, previniendo, -por ejemplo en Java- la omisión accidental de las llaves, lo que podría alterar de manera drástica el significado semántico y el comportamiento dinámico del programa durante su ejecución. En lenguajes como Java además, cualquiera de las sentencias que un bloque contiene puede ser sustituida, a su vez, por un bloque, lo que se conoce como **bloques anidados**. La anidación exige que cualquier bloque interno se cierre de forma completa mediante su llave correspondiente antes de que pueda cerrarse el bloque exterior que lo contiene.

```
{
    int a = 10;
    {
        int b = 20;
        System.out.println(a);
        System.out.println(b);
    }
}
```

Cada vez que abrimos una llave `{` y cerramos otra `}`, no solo estamos estructurando un grupo de instrucciones, sino que estamos delimitando un nuevo **ámbito** o *scope*. El ámbito se define como la zona o porción específica del programa en la que un determinado identificador resulta visible y, por consiguiente, puede ser accedido y utilizado en el interior de una expresión. El ámbito determina tanto la visibilidad de los identificadores como la vida útil o tiempo de persistencia física de las variables en la memoria del ordenador. Las variables locales —aquellas que se definen dentro de un método o en cualquier bloque de código intermedio— se crean en la memoria en el momento en que el flujo de control entra en su bloque y se destruyen en cuanto el control del programa abandona dicho bloque. Podemos pensar en el ámbito como el espacio del programa en el que existe una declaración y puede ser utilizada mediante su identificador. Cuando una variable deja de pertenecer al ámbito en el que fue declarada, no significa necesariamente que el dato haya desaparecido de forma inmediata de la memoria; significa que, desde ese punto del programa, esa variable ya no puede ser utilizada mediante ese nombre.

> **Ámbito**: región del programa dentro de la cual determinadas declaraciones son válidas y pueden ser utilizadas.

Dentro de un bloque de código, una variable local solo es válida y utilizable desde el momento exacto en que se realiza su declaración formal hasta que se alcanza la llave de cierre del bloque en el que reside. Intentar hacer uso de una variable o asignarle un valor en una línea anterior a su definición provocará un error insalvable detectado por el compilador en tiempo de compilación. Las variables locales no se inicializan por defecto y no se exige asignarles un valor inicial en la sentencia de su declaración, pero se bloqueará la compilación y emitirá un mensaje de error si el programa intenta leer o utilizar la variable local dentro de una expresión sin que se le haya asignado previamente un valor de forma explícita y segura.

Las reglas que rigen la visibilidad de los identificadores se vuelven sumamente precisas: el bloque exterior envuelve por completo al bloque interno, lo que significa que cualquier identificador declarado en el ámbito externo es perfectamente visible y accesible para todas las instrucciones que se ejecutan dentro del bloque anidado. Consideremos el siguiente ejemplo:

```
{
    int edad = 20;
    System.out.println(edad);
}
```

La variable edad ha sido declarada dentro del bloque y podemos utilizarla mientras permanezcamos dentro de ese ámbito. Sin embargo, una vez terminado el bloque, esa variable deja de estar disponible:

```
{
    int edad = 20;
}

System.out.println(edad);
```

La segunda sentencia no es válida porque edad pertenece al bloque en el que fue declarada. Fuera de él, su identificador ya no puede utilizarse.

Suiguiendo con el ejemplo que usábamos para los bloques anidados:

```
{
    int a = 10;
    {
        int b = 20;
```

```
            System.out.println(a);
            System.out.println(b);
        }
    }
```

el bloque interior tiene acceso a la variable a porque a ha sido declarada en un ámbito exterior que contiene al bloque interior. En cambio, `b` pertenece al bloque interior y no puede utilizarse desde el bloque exterior después de terminar dicho bloque.

Podemos representar esta relación de una forma sencilla:

```
ámbito exterior
│
├── a
│
└── ámbito interior
    │
    └── b
```

Desde el ámbito interior podemos utilizar las declaraciones disponibles en él y las de los ámbitos exteriores que lo contienen. Pero desde un ámbito exterior no podemos utilizar una declaración que solo exista dentro de uno de sus bloques interiores. Esta regla permite que diferentes partes de un programa utilicen nombres sin interferir innecesariamente entre sí. Podemos declarar una variable que solo necesitemos durante una parte concreta de un algoritmo y limitar su uso a ese lugar. El ámbito viene determinado por las reglas sintácticas del lenguaje y por el lugar donde se realiza la declaración.

Flujo de control

Hasta ahora hemos visto nuestros programas como algo estrictamente lineal. El ordenador se ha limitado a actuar como un ejecutor pasivo que procesa una instrucción tras otra, siguiendo el orden en el que las sentencias fueron escritas. Este esquema, aunque resulta intuitivo y constituye la base de cualquier proceso algorítmico, presenta una limitación operativa: no puede adaptarse a la variabilidad del mundo real. No basta con conocer las instrucciones que debe realizar un programa: también es necesario determinar **en qué orden deben ejecutarse**.

En los programas más sencillos, las instrucciones se ejecutan una detrás de otra, siguiendo el orden en el que aparecen escritas. Por ejemplo, si un programa contiene tres instrucciones:

```
instrucción 1;
instrucción 2;
instrucción 3;
```

la ejecución comienza por `instrucción 1`. Cuando esta termina, se ejecuta `instrucción 2` y, finalmente, `instrucción 3`.

Este comportamiento parece evidente, pero es importante observar que **el orden de las instrucciones forma parte de la solución del problema**. No siempre es posible cambiar el orden de dos instrucciones sin modificar el resultado del programa. Una instrucción puede necesitar que otra se haya ejecutado previamente para disponer de los datos que necesita. Por ejemplo, para calcular el área de un rectángulo es necesario disponer primero de su base y de su altura antes de realizar la multiplicación:

```
int base = 8;
int altura = 5;
int area = base * altura;
System.out.println("Área: " + area);
```

La asignación de `area` no puede hacerse antes de las asignaciones de `base` y `altura`, ya que necesita los valores almacenados en esas variables. El orden de ejecución, por tanto, no es arbitrario.

Concepto de flujo de control

El **flujo de control** de un programa es el orden en el que se ejecutan las instrucciones que lo forman. En un sentido figurado, podemos imaginar que la computadora se encuentra bajo el control exclusivo de una única sentencia en cada momento; una vez que esa sentencia ha sido ejecutada, el control se transfiere o «cede» a la sentencia siguiente (como los atletas se entregan el testigo en una carrera de relevos).

Flujo de control: Orden en el que se ejecutan las instrucciones de un programa.

Reiteramos lo importante que es distinguir entre el orden en el que las instrucciones están escritas y el orden en el que se ejecutan. En los programas sencillos que hemos escrito hasta ahora ambos coinciden, pero esto no tiene por qué suceder siempre. Pensemos, por ejemplo, en cómo podemos preparar un bocadillo: necesitamos pan y lo que queramos comer; necesitaremos cortar un trozo de pan de tamaño apropiado; cortar dicho trozo de forma longitudinal, aproximadamente, por la mitad; disponer el alimento entre las dos partes obtenidas por la zona de la miga. No tendría sentido intentar realizar estas acciones en cualquier orden: algunas dependen necesariamente de que otras se hayan realizado previamente.

De manera predeterminada u ordinaria, y de la forma que lo hemos visto hasta ahora, el flujo de c0ntro es lineal. Esto significa que las instrucciones se ejecutan una vez cada una, una detrás de otra, en el orden en el que aparecen en el programa. Sin embargo, los problemas el software debe resolver no siempre pueden expresarse de esta manera, En la mayoría de problemas, debe ser capaz de reaccionar de manera flexible, tomando desvíos, bifurcaciones o repitiendo tareas basándose en datos que pueda determinar durante su ejecución. Por tanto, el flujo de control de un programa puede seguir diferentes caminos dependiendo de las instrucciones que se hayan utilizado para construirlo.

Para lograr esta ejecución no secuencial, los lenguajes de programación proporcionan herramientas sintácticas especiales denominadas **estructuras de control**.

Estructuras de control

Una **estructura de control** es una sentencia diseñada específicamente para alterar el flujo ordinario de la aplicación, transfiriendo de manera deliberada el control del programa a una instrucción distinta de aquella que físicamente vendría a continuación en el archivo de texto. De acuerdo con el teorema del programa estructurado de Böhm–Jacopini, cualquier algoritmo puede ser implementado combinandosólo tres tipos de estructuras de control fundamentales:

- **La estructura secuencial o secuencia:** Representa la ejecución lineal por defecto, donde las sentencias se ejecutan una después de otra en el orden en que aparecen.
- **La estructura condicional o de selección:** Permite decidir, en función de si alguna condición es verdadera o falsa, qué instrucciones ejecutar.
- **La estructura repetitiva o iteración:** Permite que un conjunto estructurado de instrucciones se ejecute varias veces mientras se cumpla una determinada condición.

Adicionalmente, los lenguajes modernos de alto nivel añaden formas más avanzadas de gobernar este flujo lógico. Un ejemplo de ello, como avanzábamos en el tema anterior, son las llamadas a métodos: funcionan como pequeñas estructuras que detienen temporalmente la secuencia actual para delegar la ejecución en un subprograma específico, regresando al punto de origen una vez completada la tarea.

El dominio del **flujo de control** y la capacidad para trazar la ejecución del código constituyen el pilar fundamental sobre el que se construye un software eficiente, predecible y libre de fallos.

Secuencia

La **estructura secuencial** constituye el punto de partida y la forma más elemental de organizar las instrucciones dentro de un programa. Una secuencia representa un conjunto ordenado de una o más sentencias que la máquina ejecuta, una tras otra, siguiendo el mismo orden en el que están dispuestas en el código fuente. La ejecución comienza en la primera instrucción y continúa con la siguiente hasta alcanzar la última. Por ejemplo:

```
int a = 10;
int b = 20;
int suma = a + b;
System.out.println(suma);
```

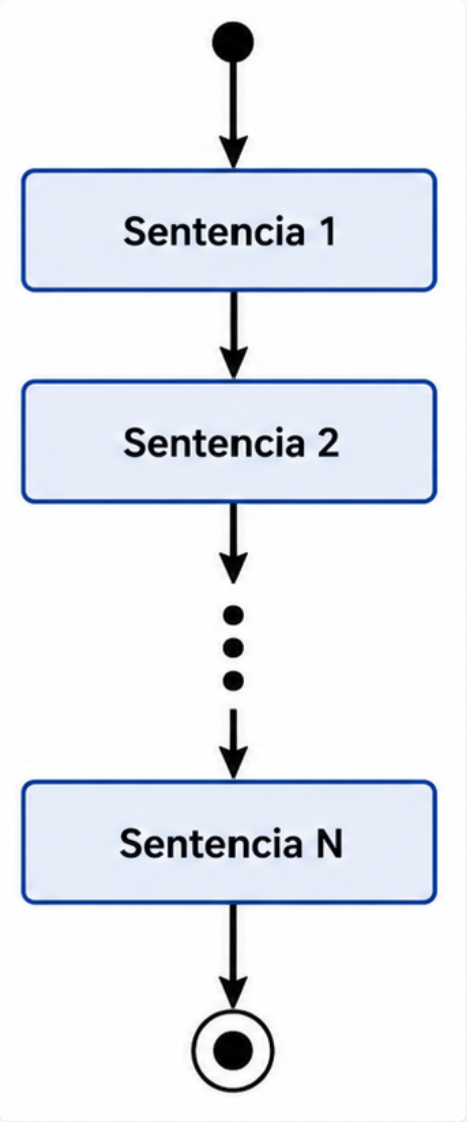
La ejecución de este fragmento sigue una secuencia perfectamente determinada:

1. Se declara la variable `a` y se le asigna el valor `10`.
2. Se declara la variable `b` y se le asigna el valor `20`.
3. Se declara la variable `suma`, se calcula la suma de `a` y `b` y se almacena el resultado en `suma`.
4. Se muestra el valor de `suma`.

Cada instrucción se ejecuta exactamente una vez y siempre después de la anterior y antes de la siguiente. El flujo de control avanza, por tanto, **de principio a fin**. Esto puede parecer una característica demasiado sencilla como para necesitar una estructura específica. Sin embargo, la secuencia es fundamental porque **cualquier algoritmo está formado, en última instancia, por operaciones que deben ejecutarse en un determinado orden**.

La secuencia puede representarse gráficamente mediante una sucesión de bloques conectados que indican el sentido del flujo de ejecución:

```
┌────────────────────┐
│ int a = 10;        │
└──────────┬─────────┘
           ↓
┌────────────────────┐
│ int b = 20;        │
└──────────┬─────────┘
           ↓
┌────────────────────┐
│ int suma = a + b;  │
└──────────┬─────────┘
           ↓
┌────────────────────┐
│ System.out.println │
│ (suma);            │
└────────────────────┘
```



Desde el punto de vista de la arquitectura física de la computadora, la estructura secuencial se alinea de manera natural con el ciclo de búsqueda y ejecución de la unidad de control de la CPU. La computadora opera leyendo secuencialmente las direcciones de memoria donde se almacenan las instrucciones, decodificando cada orden en señales de control y enviándolas a los componentes del sistema para ejecutar la tarea. Seguir una estructura secuencial es equivalente a interpretar las notas de una partitura musical de principio a fin de forma lineal.

En la sintaxis de Java, las sentencias simples que integran una secuencia se delimitan formalmente mediante el carácter obligatorio del **punto y coma** (`;`), que actúa como el token terminador de cada acción individual. Asimismo, estas instrucciones secuenciales pueden ser agrupadas de manera colectiva dentro de un **bloque de código** delimitado por llaves `{}`, comportándose exteriormente frente al compilador como si fuesen una sola unidad de acción indivisible. Dominar la construcción rigurosa de secuencias lógicas coherentes es el requisito previo e ineludible para garantizar la robustez, el orden y la calidad de cualquier algoritmo de software.

Selección

Una vez comprendido que la estructura secuencial constituye el esqueleto sobre el cual se disponen las instrucciones simples en el orden de su escritura, que se revela del todo insuficiente para nuestros programas, es el momento de dotarlos de la capacidad de responder y adaptarse dinámicamente a diferentes situaciones. Para que una aplicación sea verdaderamente útil e inteligente, debe abandonar la rigidez de la ejecución lineal y adquirir la facultad de tomar decisiones autónomas basadas en el estado cambiante de sus datos en tiempo de ejecución. Esta capacidad de modificar el flujo de control de un programa haciendo que determinadas instrucciones se ejecuten o no, o escogiendo entre diferentes conjuntos de instrucciones, es lo que denominamos **estructura selectiva** e implementamos de manera formal en el código mediante la estructura de **selección** o condicional.

El fundamento de la selección reside en formular una proposición lógica, frecuentemente denominada aserción o **condición**. Una condición no es más que una expresión booleana —ya sea una simple comparación relacional o una compleja combinación de operadores lógicos— que, al ser procesada por el ordenador, produce de manera inequívoca uno de dos únicos valores posibles: verdadero o falso.

Condición: Proposición o sentencia lógica que, tras ser evaluada sintácticamente por el sistema, devuelve un valor de verdad (cierto o falso), sirviendo como base para la toma de decisiones en las estructuras de control.

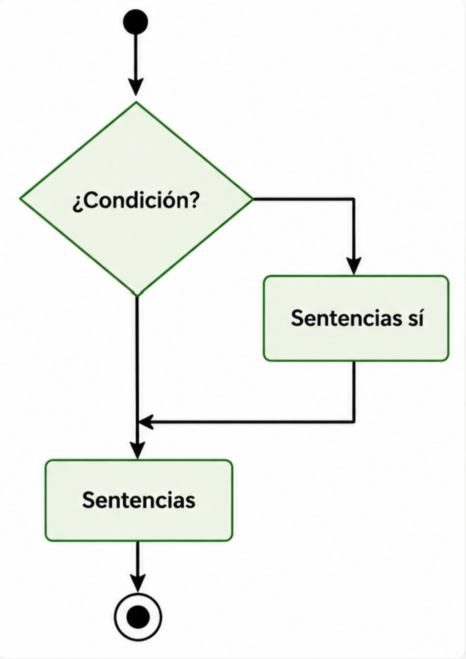
Es importante destacar que la toma de decisiones en programación no añade nuevas operaciones físicas al hardware: altera de forma dinámica el código que ejecuta la máquina. Al introducir estructuras de selección, el código que escribimos deja de ser lineal y pasa a convertirse en una red de alternativas lógicas. La computadora elegirá el flujo de control adecuado según las condiciones establecidas por el programador. Esta versatilidad no solo es la clave para la resolución de problemas prácticos complejos, sino que también representa el pilar básico para construir programas robustos, capaces de validar datos de entrada erróneos o inesperados, reaccionar de manera controlada ante fallos y comportarse de forma coherente durante la actividad ordinaria de la aplicación.

Selección simple

La **selección simple** es la estructura de decisión más elemental y fundamental de la que dispone un lenguaje de programación para alterar de forma condicionada el **flujo de control** de una aplicación. Conocida técnicamente como bifurcación condicional de una única rama, esta estructura permite al programador condicionar la ejecución de una sentencia o bloque de instrucciones a que una proposición lógica concreta resulte ser verdadera. Si, al evaluar dicha aserción, el resultado obtenido es verdadero, el ordenador detendrá momentáneamente su avance lineal para ejecutar la acción indicada en la bifurcación; por el contrario, si la condición se evalúa como falsa, la computadora la ignorará por completo y continuará la ejecución en la sentencia secuencial que se encuentre físicamente a continuación del condicional. En resumen, se ejecutará una sentencia **solo cuando se cumple una determinada condición**.

Sintácticamente, la selección simple se implementa en Java mediante el uso de la palabra reservada **`if`**, seguida de una expresión booleana encerrada entre paréntesis. La plantilla sintáctica general de esta instrucción responde al siguiente esquema:

```
Statement:
  StatementWithoutTrailingSubstatement
  IfThenStatement

IfThenStatement:
  if ( Expression ) Statement
```



Por ejemplo, supongamos que queremos mostrar un mensaje únicamente cuando una persona sea mayor de edad:

```
int edad = 20;

if ( edad >= 18 ) {
    System.out.println("Es mayor de edad");
}
```

El ordenador comienza ejecutando la asignación de `edad`. Cuando llega a la sentencia **if**, evalúa la expresión `edad >= 18`. Si el resultado es `true`, ejecuta la sentencia contenida dentro del bloque. Si el resultado es `false`, no ejecuta el bloque y continuaría con la siguiente instrucción.

- **La expresión de control o condición**: El contenido entre los paréntesis debe ser una expresión booleana (verdadero o falso). Aunque otros lenguajes puedan permitirlo, Java es fuertemente tipado y solo acepta un valor booleano: el compilador indicará que hay un error de tipo.
- **El cuerpo de la estructura**: Representa la sentencia o conjunto de sentencias que quedan supeditadas al éxito de la condición. La plantilla declara la sentencia condicional *`IfThenStatement`* introduciendo *`Statement`* para el caso en que se cumpla la condición: si el cuerpo consta de una única sentencia simple finalizada en punto y coma, la sintaxis de Java permite omitir las llaves delimitadoras; si se

requiere ejecutar una secuencia compuesta por más de una instrucción, es imperativo agruparlas dentro de un **bloque de código**.

Desde una perspectiva metodológica y de calidad del software, muchas guías de estilo modernas recomiendan utilizar siempre bloques, incluso cuando el cuerpo de la estructura tenga una única sentencia[19]. Un error clásico consiste en añadir una instrucción a un cuerpo que carece de llaves, asumiendo erróneamente que ambas quedarán bajo el control de la condición; en realidad, el compilador interpretará que solo la primera sentencia está condicionada y la segunda se ejecutará siempre.

Importante también tener en cuenta un error sintáctico sumamente común y de difícil detección: insertar accidentalmente un **punto y coma tras el paréntesis de cierre** de la condición: el compilador no emite ningún mensaje de error porque interpreta que el cuerpo es la **sentencia nula**, representada por ese punto y coma. Consideremos el siguiente código:

```
  int divisor = obtenerDivisor(); // Devuelve un entero
  double resultado = 0.0;

  // Hacemos uso de la selección simple para reaccionar ante una entrada
anómala
  if ( divisor == 0 );
      System.out.println("Advertencia: El divisor proporcionado es cero.");
```

Resultan irrelevantes el valor de `divisor` y la indentación: siempre se imprimirá la advertencia.

Selección compuesta

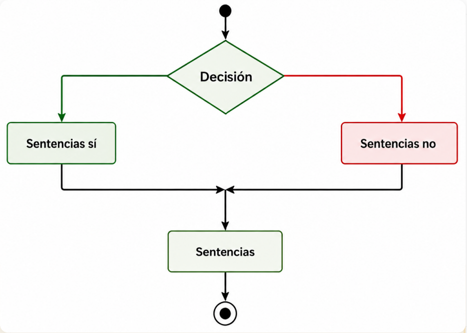
La práctica real de la resolución de problemas nos plantea con frecuencia escenarios donde es necesario elegir entre dos alternativas de acción mutuamente excluyentes. La selección simple permite actuar cuando una condición es verdadera pero, en muchos problemas, necesitamos hacer algo diferente cuando la condición es falsa. Imaginemos que, ampliando el ejemplo anterior, queremos también saber si la persona es menor de edad; en definitiva, es lo mismo que decir que no es mayor de edad:

```
  int edad = 16;

  if ( edad >= 18 )
      System.out.println("Es mayor de edad");
  if ( ! (edad >= 18) ) // edad < 18
      System.out.println("Es menor de edad");
```

---

[19] Ese no va a ser nuestro caso, valorando, precisamente, que no cometéis ese error.

Hemos escrito la condición de esa manera para significar que necesitamos replicar la sentencia condicional pero negada. Para simplificar esta cicunstancia, surge la denominada **selección compuesta** o bifurcación condicional de dos ramas.



```
Statement:
  StatementWithoutTrailingSubstatement
  IfThenStatement
  IfThenElseStatement

IfThenElseStatement:
  if ( Expression ) Statement else Statement
```

Ahora el flujo de control tiene dos caminos posibles. Si la condición es verdadera, se ejecuta el primer bloque; si es falsa, se ejecuta el segundo:

1. **Evaluación de la expresión de control**: Se evalúa la condición booleana encerrada entre los paréntesis: un valor booleano (verdadero o falso).
2. **Bifurcación del camino:**

- Si el resultado de la evaluación de la condición es verdadero, el ordenador transfiere inmediatamente el control a la primera sentencia (denominado cláusula ***then***). Una vez completada la ejecución el flujo de control continua en la primera sentencia ubicada después de la estructura condicional completa.
- Si la condición se evalúa como falsa, el flujo de control continua con la segunda sentencia (conocida como cláusula ***else***); y después. con la primera sentencia ubicada después de la estructura condicional completa.

A diferencia de la selección simple, en este caso **siempre se ejecuta una de las sentencias**. Ambas sentencias son **excluyentes**: la estructura garantiza que se procesará una o la otra, nunca ambas en una misma evaluación de la condición.

Selección simple: Estructura de control que permite ejecutar una sentencia si una determinada condición se cumple.

Selección compuesta: Estructura de control que permite escoger entre dos alternativas dependiendo de si una condición es verdadera o falsa.

Esta estructura se denomina también **decisión** porque permite escoger entre dos alternativas en función del resultado de una condición.

Observemos en la plantilla que tanto la parte asociada al `if` como la asociada al `else` son sintácticamente una `Statement`. Por tanto, ambas pueden ser una sentencia simple o un bloque de código. Que volviendo a lo que comentábamos antes, muchas guías de estilo recomiendan usar siempre llaves, cosa que, si se hace, debe hacerse en ambos casos.

Operador condicional ternario

Dentro del estudio de la **selección compuesta**, el lenguaje Java ofrece un mecanismo compacto diseñado para resolver un escenario algorítmico sumamente frecuente: la evaluación de una condición booleana para seleccionar y devolver uno de dos valores posibles. Aunque la sentencia **if-else** resuelve perfectamente cualquier bifurcación lógica, esta se concibe como una instrucción de control orientada a ejecutar bloques de código. Sin embargo, cuando la única finalidad de una bifurcaciónes asignar un valor a una variable o pasar un argumento a un método, el uso de una estructura completa puede resultar tedioso. Para estos casos, Java proporciona un **operador ternario** que permite llevar a cabo esta tarea dentro de una única expresión.

La denominación de operador ternario proviene directamente de su aridad[20]: es el único operador en Java que requiere exactamente tres operandos para formar una expresión válida. Sintácticamente, se construye combinando los símbolos del signo de interrogación (`?`) y los dos puntos (`:`), respondiendo a la siguiente plantilla sintáctica:

```
ConditionalExpression:
  ConditionalOrExpression
  ConditionalOrExpression ? Expression : ConditionalExpression
```

La secuencia de evaluación es la siguiente:

1. **Evaluación de la condición**: En primer lugar, se procesa el primer operando, *`ConditionalOrExpression`*, que debe ser una expresión booleana (verdadero o falso).
2. **Selección del resultado**: Si la condición resulta ser verdadera, el operador evalúa únicamente la primera expresión, *`Expression`*, y su resultado se convierte en el valor final de toda la expresión condicional. Por el contrario, si la condición se evalúa como falsa, el sistema evalúa exclusivamente la segunda expresión, *`ConditionalExpression`*, devolviendo su valor como resultado global.
3. **Evaluación condicional de expresiones**: De forma análoga a la evaluación en cortocircuito, el operador ternario jamás evalúa ambas expresiones de forma simultánea. Solo se procesa la correspondiente al resultado del primer operando, lo que previene la ejecución de cálculos innecesarios o con efectos laterales.

Una restricción de tipo fundamental es que las dos expresiones de retorno deben evaluar a tipos de datos compatibles entre sí y con la variable que recibirá el resultado. Para apreciar la elegancia y concisión de este operador, podemos comparar la forma de obtener el valor absoluto de un número mediante una sentencia **if-else** frente a este operador condicional:

```
int valor = -10;
int valorAbsoluto;
```

---

[20] En el análisis matemático, la aridad de un operador matemático o de una función es el número mínimo de argumentos necesarios para que dicho operador funcione, o se pueda calcular.

```
// Mediante selección compuesta (if-else)
if (valor < 0) {
    valorAbsoluto = -valor;
} else {
    valorAbsoluto = valor;
}

// Utilizando el operador ternario
int valorAbsolutoTernario = (valor < 0) ? -valor : valor;
```

El operador ternario no debe considerarse un sustituto universal de la sentencia **if-else**: es un recurso sintáctico para decisiones simples a nivel de expresión. Un error habitual cuando se está aprendiendo consiste en intentar anidar múltiples operadores ternarios (`cond1 ? exp1 : cond2 ? exp2 : exp3`) para emular múltiples caminos **if-else-if**; esto genera un código intrincado, difícil de leer y propenso a errores. La recomendación de estilo es la de priorizar siempre la claridad estructural del código sobre la brevedad de la sintaxis.

Selección múltiple

Anidación

En ocasiones, un problema no tiene únicamente dos alternativas. Es habitual encontrarnos con situaciones en las que la toma de decisiones no siempre se presenta como un conjunto de opciones simples dispuestas en un único nivel. Con frecuencia, la resolución de un problema exige evaluar condiciones secundarias que dependen estrictamente del resultado de una evaluación condicional previa. Es en estos escenarios donde cobra pleno sentido la **anidación** (*nesting*). Se considera que una estructura está anidada cuando se ubica físicamente en el cuerpo o rama de otra estructura.

Supongamos que queremos clasificar una calificación:

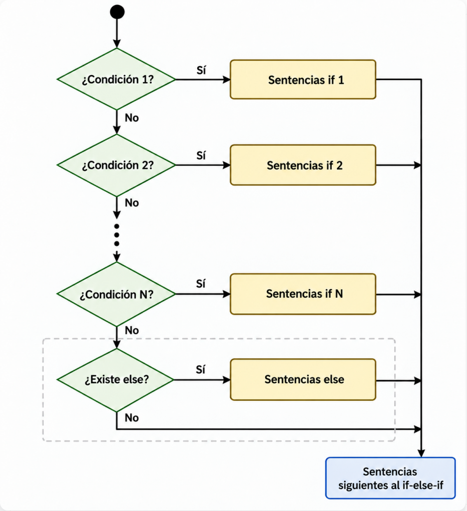
```
if (nota >= 9)
    System.out.println("Sobresaliente");
else
   if (nota >= 7)
      System.out.println("Notable");
   else
      if (nota >= 5)
        System.out.println("Aprobado");
      else
         System.out.println("Suspenso");
```

En este caso, tenemos una decisión pero, cuando la condición no se cumple, tenemos una nueva condición y esta, a su vez, otra. Esto no constituye de por sí una nueva sentencia ya que se obtiene encadenando decisiones: un **if** dentro de la alternativa **else** de otro **if**. El anidamiento de condicionales introduce un desafío sintáctico y de diseño conocido como el dilema del **else huérfano** (*dangling else*). Cuando escribimos condicionales anidados complejos puede surgir una ambigüedad aparente sobre a qué **if** concreta pertenece un determinado **if**. Para resolver esta incertidumbre el compilador aplica una regla de asociación sintáctica estricta: **un bloque else siempre se asocia con el condicional if más cercano y anterior que se encuentre dentro de su mismo bloque de código** (siempre y cuando dicho if no cuente ya con un else asociado). Recordad que el compilador ignora por completo los espacios en blanco, los retornos de línea y la indentación: alinear visualmente cada grupo **if-else** es fundamental pero, en casos en los que uno quede alejado del otro o algún **if** no tenga **else**, es mejor utilizar llaves para no tener fallos lógicos difíciles de depurar.

Selección encadenada

Reescribamos el código anterior sin anidación y escribiendo cada `if` justo a continuación del `else` en el que se anida:

```
if (nota >= 9)
    System.out.println("Sobresaliente");
else if (nota >= 7)
    System.out.println("Notable");
else if (nota >= 5)
    System.out.println("Aprobado");
else
    System.out.println("Suspenso");
```



Podemos asimilar que en este caso tenemos varias condiciones y el ordenador las comprueba en orden. Si encuentra una condición verdadera, ejecuta el bloque correspondiente y continúa después de la estructura completa. Las condiciones posteriores no se evalúan.

Esta forma de construir una selección se denomina estructura de selección encadenada o **if-else-if**. Aunque esta construcción no existe como una palabra reservada o sentencia independiente en la gramática formal del lenguaje, se presenta como un patrón de diseño y tabulación específico con entidad propia. El gran beneficio estético y metodológico de este estilo es que evita que el sangrado del código marche de forma continuada e indefinida hacia la derecha de la pantalla (*indentation march to the right*), lo que dificulta enormemente su lectura. En su lugar, el código se dispone de forma vertical y compacta, transmitiendo visualmente la idea de que estamos ante una bifurcación de múltiples caminos alternativos que se evalúan secuencialmente de arriba a abajo.

Selección múltiple con switch

Existe otro tipo de problema de selección múltiple especialmente frecuente: necesitamos comparar el valor de una única variable o expresión con una serie extensa de valores concretos y mutuamente excluyentes. Es perfectamente válido resolver este escenario con una selección encadenada **if-else-if** pero puede volverse tedioso y propenso a errores. Una alternativa más limpia y legible es la estructura de selección múltiple conocida como sentencia **switch** (o sentencia **case**, según el lenguaje).

Esta sentencia funciona, conceptualmente, como un conmutador que evalúa una única expresión de control y, en función de su resultado, deriva el **flujo de control** del programa directamente hacia el bloque de instrucciones que corresponde al valor coincidente. Por ejemplo, podemos mostrar el nombre de un día a partir de un número:

```
int dia = 3;

switch (dia) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles");
        break;
    default:
        System.out.println("Día no válido");
}
```

La sentencia **`switch`** evalúa una expresión y compara su resultado con los diferentes valores indicados mediante la palabra **`case`**. Cuando encuentra una coincidencia, comienza la ejecución de las instrucciones asociadas a ese valor. La cláusula **`default`** permite indicar qué debe ejecutarse cuando no se encuentra ninguna coincidencia.

```
StatementWithoutTrailingSubstatement:
  Block
  EmptyStatement
  ExpressionStatement
  SwitchStatement

SwitchStatement:
  switch ( Expression ) SwitchBlock

SwitchBlock:
  { {SwitchBlockStatementGroup} {SwitchLabel :} }

SwitchBlockStatementGroup:
  SwitchLabel : {SwitchLabel :} BlockStatements

SwitchLabel:
  case CaseConstant
  default
```

```
CaseConstant:
  ConditionalExpression
```

Vamos a analizar las reglas de diseño y restricciones sintácticas que el compilador de Java impone sobre cada uno de sus componentes:

- **La expresión de control**: El valor situado entre los paréntesis, el **selector**, debe evaluar a un tipo de dato compatible con la estructura. Habitualmente, en función del lenguaje, esta expresión debe ser un tipo entero o un enumerado[21]. En Java, además[22], se permite el uso cadenas de texto (objetos de la clase `String`). Java prohíbe de forma explícita el uso de tipos numéricos de coma flotante o expresiones booleanas, aunque pueden ser válidas en otros lenguajes; en cualquier caso, no es recomendable su uso.
- **La lista de etiquetas**: Cada uno de los caminos lógicos alternativos se define mediante la palabra reservada `case` seguida de una expresión constante, denominada **etiqueta**, y el carácter de dos puntos (`:`). La restricción fundamental es que dicha etiqueta debe ser una **constante única conocida en tiempo de compilación**; no está permitido utilizar variables o condiciones lógicas. Además, no deben existir etiquetas duplicadas dentro de una misma estructura switch, garantizando así que no haya ambigüedad sobre qué camino debe tomar la ejecución.
- **La cláusula por defecto**: Esta sección, identificada por la palabra reservada `default`, actúa como un bloque de contingencia opcional. Las instrucciones contenidas en su interior solo se ejecutarán si el resultado de evaluar la expresión de control no coincide con ninguna de las constantes especificadas en las etiquetas case. Aunque su uso no es obligatorio, muchas fuentes recomiendan incluirla siempre identificar valores imprevistos o inconsistencias en los datos del sistema.

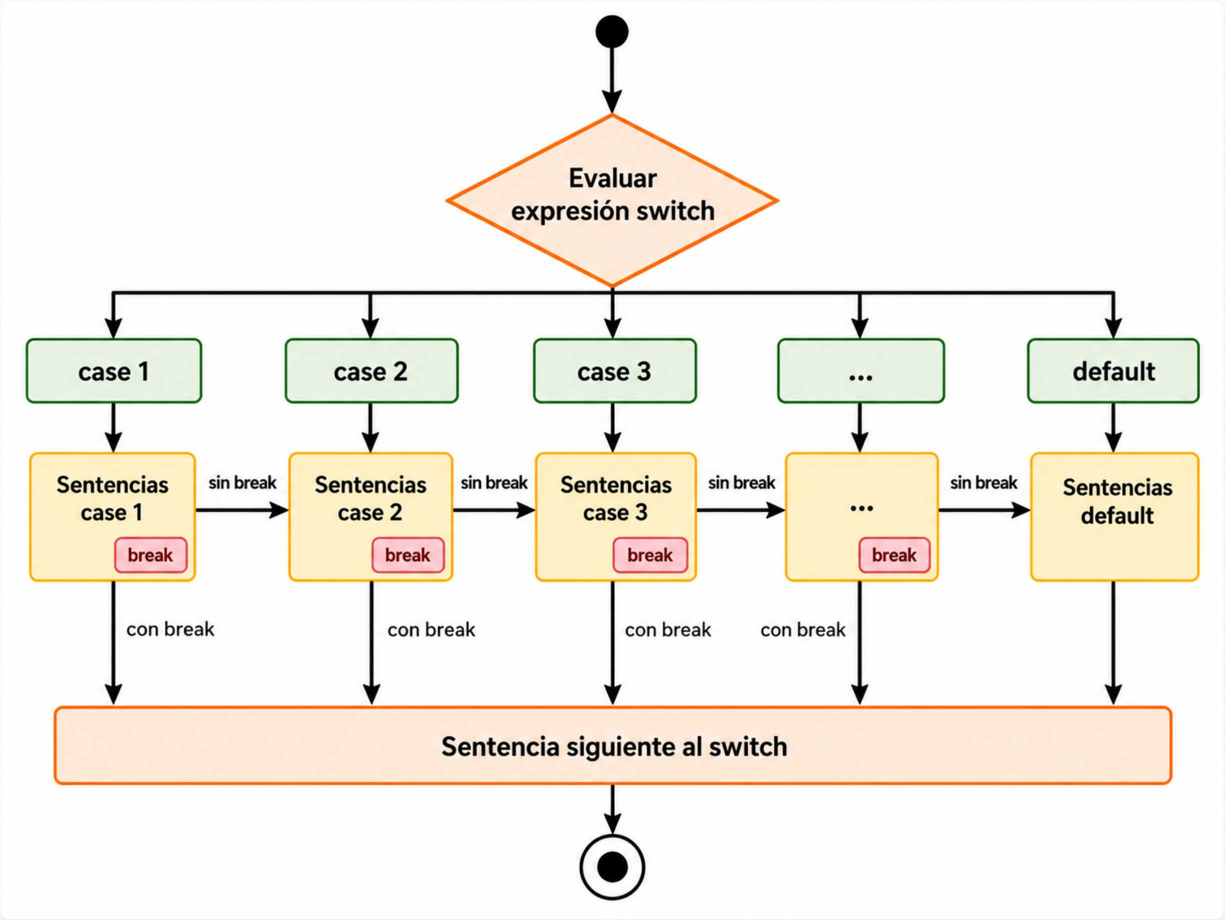
Debemos prestar especial atención a que las instrucciones de una etiqueta no están delimitadas automáticamente por unas llaves. El aspecto más crítico y que requiere mayor atención es el comportamiento conocido como **efecto de caída o cascada** (*fall-through*)[23]. Cuando se encuentra una coincidencia, el **flujo de control** realiza un salto directo a las instrucciones de esa rama. Sin embargo, a diferencia de lo que ocurre en las bifurcaciones if-else, el final del bloque no provoca la salida automática del condicional: la ejecución continuará discurriendo de forma secuencial, procesando y ejecutando las sentencias de los casos siguientes de manera ininterrumpida, sin importar que sus valores constantes no coincidan con la expresión de control.

---

[21] Son tipos de datos de usuario que se construyen definiendo un conjunto finito de valores con nombre, conocidos como constantes de enumeración. Un posible ejemplo son los días de la semana: lunes, martes, etc.

[22] A partir de la versión 7 del JDK.

[23] El término *fall-through* es característico de lenguajes herederos de C, donde la omisión del `break` se diseñó intencionadamente para permitir la ejecución agrupada de varios casos sin duplicar código.


Evaluar expresión switch
case 1
case 2
case 3
...
default
Sentencias case 1
Sentencias case 2
Sentencias case 3
...
Sentencias default
break
sin break
con break
Sentencia siguiente al switch

En el ejemplo anterior aparece además la sentencia `break`. La llamada a esta sentencia especial interrumpe de inmediato el procesamiento dentro del bloque y transfiere el flujo de control a la primera sentencia que se encuentre físicamente después de la llave de cierre de la estructura.

```
StatementWithoutTrailingSubstatement:
  Block
  EmptyStatement
  ExpressionStatement
  SwitchStatement
  BreakStatement

BreakStatement:
  break ;
```

Aunque existen lenguajes que no poseen esta característica, el efecto de caída no debe considerarse un defecto de diseño de Java, sino una potente característica sintáctica que, utilizada apropiadamente, permite agrupar múltiples etiquetas case para que desencadenen exactamente la misma acción. Consideremos el siguiente fragmento de código orientado a determinar si un día de la semana representado de forma numérica es laborable o no:

```
int diasemana = obtenerDiaDeLaSemana(); // Devuelve un entero entre 1 y 7
boolean laborable;

switch (diasemana) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        laborable = true;
        break;
    case 6:
    case 7:
        laborable = false;
        break;
    default:
        laborable = false;
}
```

En este escenario, si la variable `diasemana` vale `3` (Miércoles), el programa saltará directamente a la etiqueta `3`. Como esta cláusula carece de instrucciones y de sentencia break, la ejecución caerá secuencialmente a través de las etiquetas `4` y `5`, donde finalmente se asignará el valor `true` a la variable `laborable` antes de que la instrucción **`break`** detenga de forma segura la caída y finalice el bloque.

Desde el punto de vista del rendimiento del hardware, esta sentencia presenta una ventaja arquitectónica fundamental frente a las secuencias **if-else-if**. El compilador inspecciona las constantes de los casos y genera de forma interna una **tabla de saltos** (*jump table*)[24] en el código. Gracias a la misma, no se necesita ir comparando uno a uno todos los valores, sino que puede saltar en un solo paso al camino de ejecución exacto en función del valor obtenido. Esto hace que sea una estructura extraordinariamente eficiente cuando el número de alternativas es elevado.

Podemos tener una visión más compacta agrupando las etiquetas que dan lugar a la ejecución de las mismas sentencias; el código anterior podría haber sido escrito como:

```
switch (diasemana) {
    case 1: case 2: case 3: case 4: case 5:
        laborable = true;
        break;
    case 6: case 7:
        laborable = false;
        break;
    default:
        laborable = false;
```

---

24. La tabla de saltos evalúa el valor de entrada y dirige la ejecución instantáneamente a la dirección de memoria adecuada.

```
}
```

En el siguiente apartado estudiaremos la otra gran estructura de control: la **repetición**, que permitirá que un mismo conjunto de instrucciones pueda ejecutarse varias veces.

Switch expressions

En Java 12 se introdujo la posibilidad de usar esta construcción como una expresión, denominada **switch expressions**, que se implantó como algo definitivo en Java 14, y que modifica este modo tradicional de la sentencia para hacerla más limpia y compacta. Podemos modificar ligeramente la plantilla sintáctica para que no sea necesario escribir `case` con cada etiqueta, asociando una lista de etiquetas a una sola palabra:

```
SwitchLabel:
  case CaseConstant {, CaseConstant}
  default
```

Lo que hace que el código pueda escribirse como:

```
switch (diasemana) {
    case 1, 2, 3, 4, 5:
        laborable = true;
        break;
    case 6, 7:
        laborable = false;
        break;
    default:
        laborable = false;
}
```

Análogamente, podemos prescindir de `break` usando el operador flecha (->):

```
switch (diasemana) {
    case 1, 2, 3, 4, 5 -> laborable = true;
    case 6, 7 -> laborable = false;
    default -> laborable = false;
}
```

Para ello, de nuevo, modificamos ligeramente nuestra plantilla:

```
SwitchBlock:
  { SwitchRule {SwitchRule} }
  { {SwitchBlockStatementGroup} {SwitchLabel :} }
```

```
SwithRule:
  SwitchLabel -> Expression ;
  SwitchLabel -> Block
```

Sin embargo, hasta aquí aún no es una expresión:

```
UnaryExpressionNotPlusMinus:
  CastExpression
  SwitchExpression

SwitchExpression:
  switch ( Expression ) SwitchBlock
```

Y ya podemos volver a escribir el código, esta vez como expresión:

```
boolean laborable = switch (diasemana) {
    case 1, 2, 3, 4, 5 -> true;
    default -> false;
}
```

¿Qué sucede si en una de las sentencias debemos ejecutar un bloque de código en lugar de una única sentencia?¿Cómo puede saber Java cuál es el valor que debe retornar desde la expresión a la variable?

```
StatementWithoutTrailingSubstatement:
  Block
  EmptyStatement
  ExpressionStatement
  SwitchStatement
  BreakStatement
  YieldStatement

Yield Statement:
  yield Expression ;
```

Dentro del bloque escribiremos la sentencia **yield** que incluye dicha palabra contextual, seguida de una expresión, el valor a devolver por la sentencia `switch`.

```
int precioFinal = switch ( dia ) {
    case 1, 2, 3, 4 -> 5; // sentencia

    case 5, 6, 7 -> { // bloque
        int precioBase = 10;
        int impuesto = 2;
        int total = precioBase + impuesto;
```

```
            yield total; // Devuelve el valor y sale del switch
        }
    };
```

Repetición

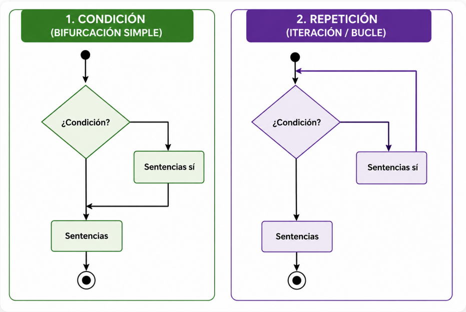
Hasta el momento hemos analizado cómo el flujo de control de un programa puede ejecutarse de manera lineal o bifurcarse. Estas dos estructuras nos permiten resolver muchos problemas pero si la capacidad de cómputo estuviera restringida a procesar cada línea de código una única vez, cuando se alcanza su posición dentro del flujo de control del programa, la utilidad del software se vería drásticamente reducida. La verdadera potencia de las computadoras modernas no reside sólo en su capacidad para almacenar datos o tomar decisiones puntuales, sino en su asombrosa velocidad para ejecutar la misma secuencia de instrucciones tantas veces como sea necesario sin cometer errores por fatiga o distracción.

Supongamos, por ejemplo, que queremos mostrar en pantalla los números del 1 al 5. Una forma de hacerlo consistiría en escribir cinco sentencias de salida:

```
System.out.println(1);
System.out.println(2);
System.out.println(3);
System.out.println(4);
System.out.println(5);
```

El programa funciona correctamente, pero resulta evidente que estamos repitiendo la misma operación. Si quisiéramos mostrar los números del 1 al 1000, escribir mil sentencias no sería una solución razonable. Y el problema sería todavía mayor si el número de repeticiones no fuese conocido de antemano, sino que dependiese de los datos que se obtienen durante la ejecución del programa. El verdadero problema que debemos resolver como programadores es cómo expresar esa repetición sin tener que escribir varias veces las mismas instrucciones.

La **estructura repetitiva**, también denominada, **bucle** o ***loop***, permite precisamente indicar que una misma sentencia o un conjunto de sentencias debe ejecutarse varias veces dentro de un programa. En lugar de copiar las instrucciones tantas veces como queramos ejecutarlas, escribimos una sola vez el conjunto de operaciones y establecemos las condiciones que determinan su repetición.



Desde un punto de vista formal, un bucle es una construcción sintáctica que permite ejecutar un bloque de sentencias de forma reiterada mientras se mantenga verdadera una determinada proposición lógica, conocida como **condición de permanencia** o condición de control. Cada una de las ejecuciones individuales del bloque de código contenido en el interior del bucle recibe el nombre de **iteración**. De este modo, la repetición transforma una secuencia finita de instrucciones en un proceso capaz de iterar sobre colecciones de información o mantener una aplicación en ejecución continua a la espera de eventos del usuario.

Podemos entender la repetición como una modificación del flujo de control que ya conocemos. En una estructura secuencial, después de ejecutar una sentencia el control pasa a la siguiente; en una selección, el resultado de una condición determina qué camino debe seguir la ejecución. En una repetición aparece una posibilidad nueva: después de ejecutar unas sentencias, el flujo de control puede regresar a una posición anterior del programa para volver a ejecutarlas. De esta forma, el flujo de control deja de avanzar únicamente hacia delante: una parte del programa puede recorrerse varias veces.

La característica fundamental de cualquier repetición es, por tanto, que existe un **conjunto de instrucciones que puede ejecutarse más de una vez**. En cada iteración se realiza nuevamente el mismo conjunto de operaciones, aunque los valores sobre los que trabajan dichas operaciones pueden haber cambiado desde la iteración anterior.

Repetición: Estructura de control que permite ejecutar varias veces una misma sentencia o conjunto de sentencias, de acuerdo con una determinada condición o criterio de repetición.

Iteración: Cada una de las ejecuciones completas del cuerpo de una estructura repetitiva.

Para que una estructura repetitiva funcione de manera correcta, predecible y segura dentro de un algoritmo, debe articularse internamente alrededor de cuatro componentes fundamentales:

- Inicialización: Consiste en establecer el estado inicial de las variables que van a gobernar el comportamiento del bucle antes de que el flujo de control alcance la estructura por primera vez.
- Condición de control o permanencia: Es una expresión booleana que se evalúa en cada iteración. Mientras esta condición sea verdadera, el bucle continuará ejecutando iteraciones; en el instante exacto en que se evalúe como falsa, la repetición se interrumpirá y el flujo de control saltará a la primera sentencia ubicada tras el bucle.

- Cuerpo del bucle: Es el bloque de sentencias que contiene las operaciones destinadas a ejecutarse en cada iteración.
- Actualización o modificación del estado: Es la instrucción (o grupo de instrucciones) dentro del cuerpo del bucle que actualiza el valor de las variables de control. Su función es garantizar que, tras un número finito de iteraciones, la condición de permanencia deje de cumplirse.

La omisión o la incorrecta formulación del mecanismo de actualización da lugar a uno de los fallos más graves y comunes en programación: el bucle infinito. Un bucle infinito se produce cuando la condición de permanencia nunca llega a evaluarse como falsa, provocando que la máquina quede atrapada en una repetición ininterrumpida que consume recursos del sistema (CPU y memoria) y bloquea la respuesta de la aplicación. Por consiguiente, el diseño de cualquier bucle exige garantizar que el proceso repetitivo concluirá en un tiempo finito.

Este es la idea esencial de todos los bucles, aunque todavía no nos dice cómo se construyen. Lo que cambia de un tipo de repetición a otro es precisamente la forma de expresar ese criterio de repetición, la manera de controlar las iteraciones y el momento en el que se comprueba si debemos continuar o abandonar el bucle.

A partir de aquí estudiaremos las diferentes formas de construir estas estructuras en Java. Comenzaremos por la sentencia `while`, que nos permitirá observar con claridad el mecanismo básico de una repetición y, a partir de él, comprender las distintas situaciones en las que podemos necesitar controlar un bucle para la construcción de algoritmos eficientes, estructurados y de alto rendimiento.

Sentencia while

La sentencia **while** recibe su nombre del inglés *while*, que podemos traducir como «mientras», y constituye la estructura repetitiva más elemental, flexible y conceptualmente pura de las que proporciona el lenguaje Java. Su funcionamiento resulta especialmente sencillo si lo relacionamos con la selección simple que acabamos de estudiar: en ambos casos se evalúa una condición y, según el resultado, se decide qué instrucciones debe ejecutar el ordenador. La diferencia fundamental es que, en una selección, una vez ejecutadas las instrucciones de la alternativa correspondiente el flujo continúa hacia delante; en una repetición, si la condición lo permite, el flujo de control **regresa al comienzo de la estructura para volver a ejecutar las mismas instrucciones**.

Sintácticamente, la sentencia `while` es una nueva clase de *`Statement`*:

```
Statement:
  StatementWithoutTrailingSubstatement
  IfThenStatement
  IfThenElseStatement
  WhileStatement

WhileStatement:
  while ( Expression ) Statement
```

Como podemos observar, la estructura de la sentencia **while** es muy similar a la de **if**. En ambos casos encontramos la palabra reservada correspondiente, una expresión encerrada entre paréntesis y, finalmente, una sentencia:

```
if     ( expresión ) sentencia
while  ( expresión ) sentencia
```

La diferencia no está, por tanto, en la forma general de la estructura, sino en el comportamiento del flujo de control. En la selección, la sentencia asociada a la condición se ejecuta como máximo una vez. En la repetición, esa misma sentencia puede ejecutarse ninguna, una o muchas veces dependiendo del valor de la condición en cada momento.

Una característica importante que debemos observar en este esquema es que **la condición se comprueba antes de ejecutar el cuerpo del bucle**. Si resulta verdadera, se ejecutan las sentencias que forman dicho cuerpo y, al finalizar, el flujo de control vuelve a evaluar la condición. Si sigue siendo verdadera, las sentencias vuelven a ejecutarse. Este proceso continúa hasta que, en alguna de las comprobaciones, la condición resulta falsa. Esto es lo que se conoce como **estructura repetitiva precondicional**, bucle de prueba previa, bucle de precondición o *pre-test loop*.

Esto explica una propiedad importante de `while`: **el cuerpo del bucle puede no ejecutarse ninguna vez**. Si la condición es falsa cuando el flujo de control alcanza por primera vez la sentencia **while**, el ordenador no ejecuta ninguna de las instrucciones de su cuerpo y continúa directamente con la primera sentencia situada después del bucle.

Supongamos, por ejemplo, que queremos mostrar los números del 1 al 5. Ya hemos visto que escribir una sentencia de salida para cada número permite resolver el problema, pero que esa solución obliga a repetir innecesariamente la misma instrucción. Mediante una repetición podemos escribir una sola vez la instrucción que debe ejecutarse y hacer que el ordenador vuelva a ella mientras se cumpla una condición:

```
int numero = 1;

while ( numero <= 5 ) {
    System.out.println(numero);
    numero = numero + 1;
}
```

Veamos con detenimiento qué sucede durante la ejecución. En primer lugar, se declara la variable `numero` y se le asigna el valor `1`. Cuando el flujo de control alcanza la sentencia **while**, el ordenador evalúa la expresión `numero <= 5`. Como en ese momento la expresión vale `true`, ejecuta el cuerpo del bucle. La primera iteración muestra el valor `1` y, después, modifica el valor de `numero`, que pasa a ser `2`. Al terminar el cuerpo, el flujo de control no continúa todavía con la siguiente sentencia del programa, sino que **regresa a la condición del while**. Ahora se vuelve a evaluar `numero <= 5`. Como `numero` vale `2`, la condición continúa siendo verdadera y se produce una nueva iteración. El mismo proceso se repite para los valores `3`, `4` y `5`. Después de la quinta iteración, la variable `numero` adquiere el valor `6`. El ordenador vuelve a evaluar la condición:

```
condición: numero <= 5
¿ 6 <= 5 ? --> false
```

En este momento la condición ya no se cumple. El cuerpo del bucle no se vuelve a ejecutar y el flujo de control continúa con la primera sentencia situada después de la estructura `while`. Podemos representar las sucesivas comprobaciones mediante una tabla:

numero	condición	acción
1	1 <= 5 → sí	ejecutar cuerpo
2	2 <= 5 → sí	ejecutar cuerpo
3	3 <= 5 → sí	ejecutar cuerpo
4	4 <= 5 → sí	ejecutar cuerpo
5	5 <= 5 → sí	ejecutar cuerpo
6	6 <= 5 → no	salir del bucle

El ejemplo anterior nos permite observar que una repetición necesita algo más que una condición. Para que el bucle pueda terminar, **algún elemento del programa debe cambiar entre una iteración y la siguiente de manera que, finalmente, la condición deje de cumplirse**. En nuestro caso, ese elemento es la variable `numero`, cuyo valor aumenta en cada iteración.

Esta circunstancia resulta esencial para comprender el funcionamiento de cualquier bucle. Si durante la ejecución del cuerpo ninguna de las circunstancias que intervienen en la condición puede cambiar, es posible que la condición permanezca indefinidamente verdadera. En ese caso, el programa continuaría ejecutando el mismo conjunto de instrucciones sin alcanzar nunca la sentencia siguiente. Eliminemos, simplemente, `numero = numero + 1;`:

```
int numero = 1;

while (numero <= 5)
    System.out.println(numero);
```

En este caso, `numero` conserva siempre el valor `1`. Por tanto, la condición `numero <= 5` será siempre verdadera y el ordenador continuará mostrando el número `1` una y otra vez. Hemos construido un **bucle infinito**.

Bucle infinito: Repetición cuya condición de continuidad permanece verdadera indefinidamente, de manera que el flujo de control no llega a abandonar el bucle.

Los bucles infinitos no son necesariamente consecuencia de un error sintáctico. El programa anterior, desde el punto de vista de las reglas del lenguaje Java, es correcto y el compilador no tiene por qué advertirnos de que nunca terminaremos la repetición. Se trata de un problema relacionado con la lógica del algoritmo y, por tanto, tendremos que ser capaces de detectarlo durante su diseño o mediante la depuración del programa.

Debemos prestar también atención a una diferencia importante respecto de una secuencia ordinaria cuando se ejecuta un bloque. Las sentencias que forman el cuerpo del bucle no deben interpretarse como acciones que se ejecutan una sola vez en el lugar en el que están escritas. Constituyen el **conjunto de instrucciones que se repite cada vez que la condición resulta verdadera**. Como consecuencia, la posición de una sentencia dentro del cuerpo puede ser relevante para el funcionamiento del algoritmo. En el ejemplo anterior, primero mostramos el número y después modificamos su valor. Si invirtiéramos las dos instrucciones, el resultado sería diferente:

```
while (numero <= 5) {
    numero = numero + 1;
    System.out.println(numero);
}
```

Ahora la primera iteración mostraría el valor `2` en lugar del `1`. Esto nos recuerda una idea que ya apareció al estudiar las estructuras secuenciales: **el orden de las instrucciones forma parte del algoritmo**. En una repetición esta importancia se multiplica porque el mismo orden vuelve a aplicarse en cada iteración.

Con todo lo explicado en mente, es necesario destacar las siguientes restricciones de diseño:

- **La expresión de control o condición de permanencia:** El contenido delimitado por los paréntesis debe ser una expresión que produzca un valor de tipo booleano. Al igual que ocurre en las estructuras de selección, Java prohíbe el uso de valores enteros o referencias como criterios de control directo.
- **El cuerpo del bucle:** Comprende la sentencia (o bloque) destinada a repetirse en cada ciclo.

Podemos resumir el funcionamiento de **while** mediante la siguiente secuencia:

1. **Evaluación de la condición *pre-test*:** El ordenador evalúa la expresión booleana.
2. **Decisión y ejecución de la iteración:**
    - Si el resultado de la evaluación es verdadero, se ejcuta el cuerpo del bucle.

    - Si el resultado es falso, la estructura repetitiva se abandona.
3. **Retroceso del flujo de control (*loopback*):** Al finalizar el cuerpo del bucle, el flujo de control regresa al paso 1.

La contrucción del código debe garantizar la presencia de tres elementos metodológicos indispensables: la **inicialización** de las variables de control, la **comprobación** de la condición en el encabezado y la **actualización explícita** del estado dentro del cuerpo. La ausencia de la sentencia de actualización —o una formulación defectuosa— conducirá a un **bucle infinito**.

Asimismo, como ya se indicó para la condición, es un error sintáctico sumamente sutil y extendido la colocación accidental de un **punto y coma** inmediatamente después del paréntesis de cierre: el compilador no detectará ningún fallo de sintaxis, pero interpretará que el cuerpo del bucle está constituido únicamente por la **sentencia nula** representada por ese punto y coma. Si la condición inicial es verdadera, la aplicación entrará en un bucle infinito silencioso y transparente.

Bucles controlados por condición

La sentencia **while** que acabamos de estudiar es un ejemplo de **bucle controlado por condición**. El número de iteraciones viene determinado por el resultado de una **condición lógica** que se comprueba durante la ejecución. Dentro de las estructuras repetitivas, los **bucles controlados por condición** representan una categoría fundamental de algoritmos donde el número exacto de iteraciones no se conoce con anterioridad a la entrada del bucle.

La sentencia **while** presenta precisamente este comportamiento: antes de cada iteración se comprueba una condición y el cuerpo se ejecuta mientras esta sea verdadera. Por ello recibe también el nombre de **bucle de prueba previa** o **bucle de entrada** (*pre-test loop*): la condición se comprueba antes de entrar en el cuerpo del bucle.

> **Bucle controlado por condición:** Estructura repetitiva en la que la continuidad de las iteraciones depende del resultado de una condición que se evalúa durante la ejecución.

Los bucles controlados por condición son la herramienta estándar para gestionar entradas interactivas de usuario, procesar flujos de datos de longitud variable o ejecutar algoritmos de búsqueda, por ejemplo. Dependiendo del mecanismo utilizado para señalar la finalización de las repeticiones, estos bucles se organizan en tres estrategias de diseño principales:

- **Bucles controlados por valor centinela (*sentinel-controlled loops*):** Se emplean cuando un programa debe leer y procesar una secuencia indeterminada de datos procedentes de un flujo de entrada (como el teclado). En este modelo, una variable denominada **centinela** actua como señal de terminación para indicar al programa que la introducción de información ha finalizado. El centinela será un dato claramente distinguible que no pueda confundirse con un valor válido dentro del dominio del problema como introducir la cadena `FIN`.

- **Bucles controlados por bandera o indicador (*flag-controlled loops*):** La repetición es gobernada por una variable de tipo booleano denominada **bandera** o *flag*. La bandera se inicializa con un valor booleano adecuado, verdadero o falso, antes de emprezar y se evalúa dentro de la condición. La bandera permanece inalterada en cada iteración hasta que, en respuesta a un evento o condición específica dentro del procesamiento, una sentencia modifica su estado (booleano), provocando la salida en la siguiente comprobación del bucle.
- **Bucles controlados por fin de flujo o fichero (*EOF-controlled loops*):** Son una especialización de la lectura de información desde archivos. La condición de permanencia no depende de un valor introducido manualmente por el usuario, sino de si los datos de entrada han terminado, como si en un objeto `Scanner` hubiera elementos pendientes de leer (`hasNext()`).

Para consolidar la implementación práctica de estas técnicas, consideremos el siguiente ejemplo didáctico de un bucle controlado por valor centinela diseñado para sumar una cantidad indeterminada de números enteros introducidos interactivamente por el usuario hasta que se teclee el valor centinela `0`:

```
  import java.util.Scanner;

  public class SumaCentinela {
      public static void main ( String[] args ) {
          Scanner teclado = new Scanner(System.in);
          final int CENTINELA = 0; // Valor especial que señala el fin de los
datos
          int sumaTotal = 0;

          // 1. Lectura antes de evaluar la condición del bucle
          System.out.print("Introduce un número entero (o " + CENTINELA + "
para finalizar): ");
          int numero = teclado.nextInt();

          // 2. Condición del bucle: se itera mientras el dato leído NO sea el
centinela
          while ( numero != CENTINELA ) {
              sumaTotal = sumaTotal + numero; // Procesamiento del dato válido

              // 3. Lectura de actualización al final del cuerpo del bucle
              System.out.print("Introduce el siguiente número (o " + CENTINELA
+ " para finalizar): ");
              numero = teclado.nextInt();
          }

          System.out.println("La suma de los valores introducidos es: " +
sumaTotal);
      }
}
```

En este fragmento, la lectura previa fuera del bucle asegura que, si el usuario introduce `0` en el primer intento, la condición `numero != CENTINELA` se evaluará inmediatamente como falsa y el cuerpo no se ejecutará ni una sola vez. Si el dato es válido, se suma a la variable acumuladora y la lectura de actualización al término del bloque solicita el siguiente número, manteniendo una coherencia lógica absoluta entre la recepción del dato y su evaluación por la Máquina Virtual de Java.

Como podemos ver, hay situaciones como esta en la que no tiene sentido comprobar el dato antes de pedirlo. Una posible solución consiste en utilizar una estructura cuya comprobación se realiza después de ejecutar el cuerpo. Java proporciona para ello la sentencia **do-while**.

La sentencia do-while

La sentencia **do-while** permite ejecutar un conjunto de instrucciones y, una vez ejecutado, comprobar una condición para decidir si debemos volver a ejecutarlo.

La sintaxis de la sentencia se corresponde con la siguiente plantilla:

```
Statement:
  StatementWithoutTrailingSubstatement
  IfThenStatement
  IfThenElseStatement
  SwitchStatement
  WhileStatement
  DoStatement

DoStatement:
  do Statement while ( Expression ) ;
```

A diferencia de la sentencia **while**, la palabra reservada **do** aparece al comienzo de la estructura y la expresión de control se encuentra al final, seguida obligatoriamente por el punto y coma.

Podemos comparar ambas estructuras directamente:

```
while ( condición ) {          do {
    Sentencias                     Sentencias
}                              } while ( condición );
```

En las dos estructuras encontramos una condición que determina si debemos continuar con la repetición. La diferencia está exclusivamente en **cuándo se comprueba**.

En **while**, la condición se comprueba antes de ejecutar el cuerpo. En **do-while**, el cuerpo se ejecuta primero y la condición se comprueba después. Por ello, podemos distinguir entre: **bucle de prueba previa** y **bucle de prueba posterior**.

Bucle de prueba previa: Bucle en el que la condición que controla la repetición se evalúa antes de ejecutar el cuerpo.

Bucle de prueba posterior: Bucle en el que la condición que controla la repetición se evalúa después de ejecutar el cuerpo.

Tomamos el ejemplo anterior y lo escribimos ahora usando **do-while**:

```
  import java.util.Scanner;

  public class SumaCentinela {
      public static void main ( String[] args ) {
          Scanner teclado = new Scanner(System.in);
          final int CENTINELA = 0; // Valor especial que señala el fin de los
datos
          int sumaTotal = 0;

           do {
              // 1. Lectura antes de evaluar la condición del bucle
              System.out.print("Introduce un número entero (o " + CENTINELA +
" para finalizar): ");
              int numero = teclado.nextInt();

              sumaTotal = sumaTotal + numero; // Procesamiento del dato

              // 2. Verificación al final del cuerpo del bucle
          } while ( numero != CENTINELA );

          System.out.println("La suma de los valores introducidos es: " +
sumaTotal);
      }
  }
```

Observemos la diferencia fundamental respecto de **while**: en este caso, **la primera ejecución del cuerpo se produce siempre**, porque la condición todavía no se ha comprobado cuando entramos en la estructura. Después de ejecutar el cuerpo, se evalúa la condición. Si resulta verdadera, el flujo de control regresa al principio del cuerpo y se produce una nueva iteración. Si resulta falsa, el bucle termina y la ejecución continúa con la sentencia siguiente.

Por tanto, la elección entre **while** y **do-while** no depende de que una estructura sea capaz de realizar algo que la otra no pueda hacer. Ambas permiten repetir un conjunto de instrucciones mientras se cumpla una condición. La diferencia está en **el número mínimo de veces que puede ejecutarse el cuerpo** y en el momento en que se realiza la primera comprobación.

Podemos resumirlo de la siguiente manera:

while **do-while**

```
                                       Sentencias
                                           │
                                           ▼
     ¿Condición?                       ¿Condición?
         │                                 │
         ├── no ──▶ 0 iteraciones          ├── no ──▶ 1 iteración
         │                                 │
         └── sí ──▶ 1 o más iteraciones    └── sí ──▶ 2 o más iteraciones
```

Esta diferencia aparentemente pequeña tiene una gran importancia en el diseño de algoritmos. Cuando planteamos una repetición debemos preguntarnos, antes de escoger la estructura sintáctica, **si tiene sentido que el cuerpo pueda ejecutarse cero veces o si necesariamente debe ejecutarse al menos una vez**.

Existe otra cuestión que debemos considerar. En un bucle controlado por condición, el número de iteraciones no suele conocerse directamente al escribir el programa. El comportamiento del usuario, los datos almacenados, los resultados de cálculos anteriores o cualquier otra circunstancia que cambie durante la ejecución pueden determinar cuándo deja de cumplirse la condición.

En el caso particular de este ejemplo nos planteamos la posibilidad de que siempre se ejecute porque el valor del centinela, cero, no afecta al resultado. Tened en cuenta que si debe verificarse el dato antes de alguna acción de una iteración, la comprobación del bucle sirve como verificación.

Bucles controlados por contador

Los bucles que hemos estudiado hasta ahora están controlados directamente por una **condición**. No sabemos necesariamente cuántas veces se ejecutará el cuerpo: el programa continúa repitiéndolo mientras la condición se mantenga verdadera y termina cuando deje de cumplirse. Sin embargo, en muchos problemas, la repetición tiene una característica diferente: sabemos que una operación debe realizarse un determinado número de veces. En contraposición a los bucles controlados por condición, los **bucles controlados por contador** (denominados también **bucles determinados** o por recuento) constituyen la categoría de estructuras repetitivas en las que el número exacto de iteraciones se conoce con anterioridad a la ejecución del cuerpo del bucle.

El **flujo de control** de la repetición se confía a la evolución ordenada de una variable específica denominada **variable de control** o **contador**, establecida a un valor inicial hasta uno final. Para que un bucle controlado por contador funcione de manera precisa y determinista, la arquitectura del código debe articular de manera explícita cuatro componentes fundamentales:

1. **La inicialización del contador:** Asignación del valor numérico de partida a la variable de control antes de que el flujo de ejecución alcance la primera comprobación del bucle.
2. **La condición de permanencia o valor límite:** Expresión booleana que compara de forma continua el valor actual del contador frente al límite final del intervalo.
3. **El cuerpo del bucle:** Bloque de sentencias que realiza el trabajo útil del algoritmo en cada una de las iteraciones.
4. **La sentencia de actualización o paso:** Modificación explícita (incremento o decremento) del valor del contador al término de cada iteración, garantizando que el estado de la variable avance de manera uniforme hacia el valor final.

Contador: Variable utilizada para controlar una repetición, cuyo valor cambia de forma sistemática de una iteración a la siguiente y permite determinar cuándo debe finalizar el bucle.

Bucle controlado por contador: Estructura repetitiva en la que una variable contador determina el número de iteraciones que se realizan.

Un ejemplo muy sencillo es el que ya utilizamos para mostrar los números del 1 al 5:

```
int numero = 1;

while ( numero <= 5 ) {
    System.out.println(numero);
    numero = numero + 1;
}
```

En este caso, `numero` es el contador. Comienza teniendo el valor `1`, se utiliza para realizar la operación que nos interesa y, al terminar cada iteración, aumenta su valor en una unidad. La condición del **while** determina cuándo debemos terminar.

Aunque este ejemplo está escrito utilizando un bucle controlado por condición, podemos observar que existe una relación muy clara entre la condición y el contador. La variable `numero` adopta sucesivamente los valores `1`, `2`, `3`, `4` y `5`; cuando pasa a valer `6`, la condición deja de cumplirse.

Podemos decir, por tanto, que el contador está determinando de manera indirecta el número de iteraciones. El siguiente patrón genérico ilustra la disposición formal de un bucle controlado por contador mediante la sentencia **while**:

```
int contador = VALOR_INICIAL; // 1. Inicialización de la variable de control

while ( contador <= VALOR_LIMITE ) { // 2. Comprobación del límite de parada

    // 3. Cuerpo del bucle (procesamiento de la iteración)
```

```
      contador = contador + PASO; // 4. Actualización del contador (paso de
incremento)
  }
```

Operadores de asignación

Para trabajar cómodamente con contadores necesitamos realizar con mucha frecuencia una operación muy sencilla: aumentar o disminuir su valor en una unidad. Hasta ahora hemos escrito esa operación de forma explícita:

```
numero = numero + 1;
```

Pero Java dispone de una forma abreviada de expresar modificaciones de una variable que resulta especialmente útil al trabajar con contadores: los operadores de **asignación compuesta**, que combinan una operación aritmética con la asignación del resultado en un único token:

```
numero += 1;
```

Del mismo modo:

Asignación compuesta	Operación explícita
`contador -= 3;`	`contador = contador - 3;`
`contador *= 2;`	`contador = contador * 2;`

```
AssignmentOperator:
  (one of)
  = *= /= %= += -=
```

Sentencia for

Aunque la sentencia *while* es capaz de implementar perfectamente un bucle controlado por condición, la estructura resultante presenta tres elementos que aparecen siempre asociados:

1. la inicialización del contador;
2. la condición que determina si debemos continuar;
3. la modificación del contador al terminar cada iteración.

La inicialización aparece antes del bucle, la condición aparece en su cabecera y la actualización del contador aparece al final de su cuerpo. Los lenguajes de programación suelen proporcionar una sentencia específica para reunir estos tres elementos en un mismo lugar; en el caso de Java, esta sentencia es: **for**. En la arquitectura del lenguaje, **for** no representa una estructura de control conceptualmente distinta a las anteriores, sino que se constituye como una **forma compacta de expresar un bucle controlado**

por contador, especialmente adecuada cuando la inicialización, la condición y la modificación de la variable están claramente relacionadas.

Su estructura general puede representarse mediante la siguiente plantilla:

```
Statement:
  StatementWithoutTrailingSubstatement
  IfThenStatement
  IfThenElseStatement
  SwitchStatement
  WhileStatement
  DoStatement
  ForStatement

ForStatement:
  for ( [ForInit] ; [Expression] ; [ForUpdate] ) Statement

ForInit:
  StatementExpression

ForUpdate:
  StatementExpression

ForInit:
  StatementExpression

StatementExpression:
  Asignment
```

Para comprender la equivalencia absoluta entre ambas construcciones en Java, basta comparar el esquema formal de una sentencia `while` frente a su traducción directa a una sentencia `for`:

```
// Estructura de un bucle por recuento con la sentencia while
inicialización;
while ( condición ) {
    sentencias;
    actualización;
}

// Estructura equivalente y compacta con la sentencia for
for ( inicialización; condición; actualización )
    sentencias;
```

Sintácticamente, la cabecera de la sentencia **for** se delimita mediante paréntesis obligatorios y exige exactamente **dos puntos y coma** (`;`) para separar de forma inequívoca las tres expresiones que la integran:

1. **Sección de inicialización:** Es la primera expresión contenida en el encabezado. Se ejecuta una **única vez** antes de iniciar la primera iteración. Su propósito es inicializar la variable de control del bucle. Una característica clave de Java es que

la variable puede declararse en esta sección, de modo que su **ámbito o alcance** (*scope*) queda estrictamente restringido al cuerpo del bucle.

2. **Sección de condición:** Es la segunda expresión del encabezado y actúa exactamente como la condición *pre-test* del bucle **while**. Se evalúa antes de procesar cada iteración, ejecutandose las sentencias del cuerpo del bucle si es verdadero; en caso contrario, la repetición se interrumpe inmediatamente y el flujo de control continua en la sentencia siguiente al **for**.
3. **Sección de actualización:** Es la tercera expresión de la cabecera. Se ejecuta automáticamente al **finalizar cada iteración**, inmediatamente después de procesar la última instrucción del cuerpo del bucle y antes de retornar a la sección de condición para evaluar la siguiente iteración. Habitualmente contiene una operación de incremento o decremento.

Para apreciar la concisión y la seguridad que aporta **for** frente a **while** en bucles determinados, consideremos el ejemplo que ya veíamos:

```
for ( int contador = 1;  contador <= 5; contador += 1 ) {
    System.out.println(contador);
}
```

La estructura puede resultar inicialmente algo extraña porque concentra en una sola línea todos esos elementos que hasta ahora habíamos visto separados. Podemos seguir el ejemplo anterior paso a paso:

```
1. int contador = 1;
2. comprobar contador <= 5
3. ejecutar el cuerpo
4. ejecutar contador += 1
5. volver al paso 2
6. cuando la condición sea falsa, continuar
```

La correspondencia entre **for** y **while** nos permite comprender también que no está limitado a incrementar una variable en una unidad. La actualización puede utilizar cualquiera de las operaciones que permita modificar el contador:

```
for (int numero = 2; numero <= 10; numero += 2) {
    System.out.println(numero);
}
```

En este caso el contador comienza en `2` y aumenta de dos en dos. El algoritmo es el mismo que habríamos expresado con **while**:

```
int numero = 2;

while (numero <= 10) {
    System.out.println(numero);
    numero += 2;
}
```

También podemos utilizar decrementos:

```
for (int numero = 10; numero >= 1; numero -= 1) {
    System.out.println(numero);
}
```

Aquí el contador comienza en `10` y va disminuyendo hasta que la condición deja de cumplirse.

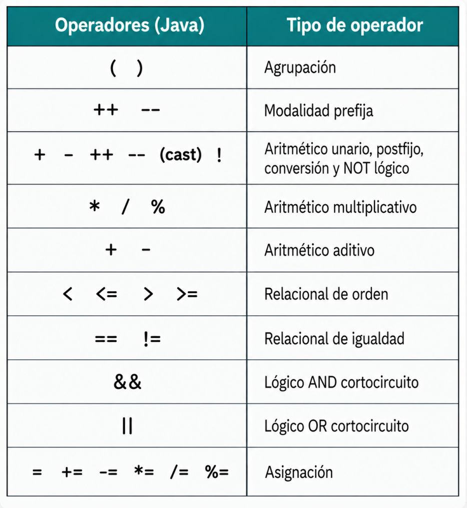
Debido a su estructura compacta, legibilidad y prevención natural contra bucles infinitos, la sentencia **for** se consolida como la construcción estándar en Java (y otros lenguajes) para recorrer intervalos numéricos o procesar iteraciones de las que conocemos el número de veces que deben ejecutarse (antes de comenzar el bucle).

Operadores de incremento y decremento

Durante el diseño de algoritmos repetitivos, resulta sumamente habitual encontrar operaciones destinadas a contar elementos o acumular totales, entre otras. Aunque estas operaciones pueden expresarse perfectamente mediante sentencias de asignación ordinarias en las que sumamos uno, la frecuencia con la que se emplean llevó a los diseñadores de lenguajes como C y Java a incorporar una serie de operadores sintácticos compactos específicamente orientados a este fin.

En primer lugar, disponemos de los **operadores unarios de incremento y decremento**:

- **Operador de incremento (++):** Suma exactamente una unidad (1) al valor actual del operando numérico sobre el que se aplica. Por consiguiente, la instrucción `x++` equivale de forma exacta a escribir `x = x+1`.
- **Operador de decremento (--):** Resta exactamente una unidad (1) al valor actual del operando numérico. De este modo, `x--` equivale a la sentencia `x = x-1`.



Operadores (Java)	Tipo de operador
( )	Agrupación
++ --	Modalidad prefija
+ - ++ -- (cast) !	Aritmético unario, postfijo, conversión y NOT lógico
* / %	Aritmético multiplicativo
+ -	Aritmético aditivo
< <= > >=	Relacional de orden
== !=	Relacional de igualdad
&&	Lógico AND cortocircuito
||	Lógico OR cortocircuito
= += -= *= /= %=	Asignación

Operadores de Java de mayor precedencia (arriba) a menor (abajo)

Estos operadores pueden utilizarse antes o después de la variable: **notación prefija** (`++x` o `--x`) y **notación posfija** (`x++` o `x--`). Cuando el operador se utiliza como una sentencia independiente, como suele ocurrir al modificar un contador, ambas formas producen exactamente el mismo resultado final: incrementar o decrementar la variable en una unidad. La diferencia aparece cuando el operador forma parte de una expresión y, por tanto, necesitamos utilizar el valor que produce:

- **Modalidad prefija (++x):** Se aplica el incremento sobre la variable **antes** de evaluar y utilizar su valor dentro de la expresión global.
- **Modalidad posfija (x++):** Se utiliza el valor actual de la variable para la expresión y, **después** de completar dicha evaluación, efectúa el incremento sobre la variable en memoria.

Ejemplo:

```
int numero = 5;
int resultado = numero++;
```

Después de estas instrucciones, `numero` vale `6`, mientras que `resultado` contiene el valor `5`: la expresión ha utilizado primero el valor que tenía el contador y después lo ha incrementado.

En cambio:

```
int numero = 5;
int resultado = ++numero;
```

deja también `numero` con el valor `6`, pero ahora `resultado` contiene `6`, porque el incremento se ha realizado antes de obtener el valor de la expresión.

La misma diferencia se aplica a los operadores de decremento.

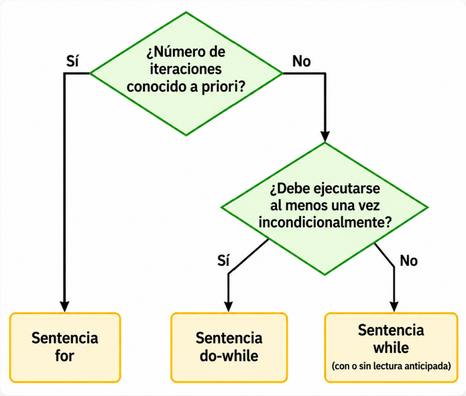
```
UnaryExpression:
  PreIncrementExpression
  PreDecrementExpression
  + Expression
  - Expression
  UnaryExpressionNotPlusMinus

PreIncrementExpression:
  ++ UnaryExpression

PreDecrementExpression:
  -- UnaryExpression

UnaryExpressionNotPlusMinus:
  PostFixExpression
  CastExpression
  SwitchExpression
```

```
PostFixExpression:
  Expression
  PostIncrementExpression
  PostDecrementExpression

PostIncrementExpression:
  PostFixExpression ++

PostDecrementExpression:
  PostFixExpression --
```

Diseño de bucles

El diseño de una estructura repetitiva constituye una de las tareas más críticas en la construcción de algoritmos robustos. Diseñar un bucle no consiste simplemente en seleccionar de forma arbitraria una palabra reservada del lenguaje, sino en analizar minuciosamente la lógica interna del problema para dar respuesta a tres interrogantes fundamentales de diseño: 1. ¿Se conoce con anterioridad a la ejecución del bucle el número exacto de iteraciones que deben realizarse? 2. ¿Existe la posibilidad de que, bajo determinadas condiciones iniciales de los datos, el cuerpo del bucle no deba ejecutarse ni una sola vez? 3. ¿Requiere el algoritmo de una **lectura adelantada** (*priming read*) antes de verificar la condición de permanencia, o, por el contrario, el proceso exige ejecutar el cuerpo de instrucciones **al menos una vez** antes de poder evaluar si se debe continuar?

A partir de la respuesta a estas preguntas, el programador debe seleccionar la estructura repetitiva más adecuada de entre las que ofrece el lenguaje (en Java, **for**, **while** y **do-while**), aplicando el patrón de diseño que garantice la máxima claridad, eficiencia y prevención defensiva contra errores lógicos.


Sí
¿Número de iteraciones conocido a priori?
No
¿Debe ejecutarse al menos una vez incondicionalmente?
Sí
No
Sentencia for
Sentencia do-while
Sentencia while
(con o sin lectura anticipada)

La sentencia do-while

Cuando el análisis del problema revela que la primera iteración de un bucle debe ejecutarse de manera obligatoria antes de poder comprobar si la repetición debe proseguir, las estructuras apropiada es utilizar una estructura repetitiva **post-test** del lenguaje: en Java, la sentencia **do-while**.

Sintácticamente, **do-while** se construye delimitando el bloque de instrucciones mediante la palabra reservada `do` (que en inglés significa «haz») y situando la cláusula de control con la palabra reservada `while` al final del bloque. Es imperativo destacar una regla sintáctica fundamental que la distingue de todas las demás estructuras de control de Java: la sentencia **do-while** exige obligatoriamente la inclusión de un punto y coma (`;`) inmediatamente después del paréntesis de cierre de la condición. La omisión de este punto y coma final constituye un error sintáctico que el compilador rechazará de inmediato.

La sentencia for

El diseño de bucles controlados por contador exige al programador prestar una especial atención a tres fuentes clásicas de error:

- **Errores por desviación en uno (*off-by-one errors*):** Ocurren cuando la condición de límite utiliza incorrectamente un operador relacional estricto (< o >) en lugar de un operador incluyente (<= o >=) o viceversa; o cuando la variable de control se inicializa en `0` en lugar de `1` (o al revés). Esto provoca que el bucle ejecute una iteración de más o una iteración de menos respecto a las previstas en el diseño del algoritmo.
- **Actualización en sentido inverso:** Si en un bucle incremental la instrucción de actualización decrementa el contador en lugar de incrementarlo (lo contrario en un bucle decremental), la variable de control se alejará progresivamente del valor límite en lugar de aproximarse a él, transformando la estructura en un **bucle infinito**.
- **Efectos laterales sobre la variable de control:** Modificar de forma arbitraria el valor del contador dentro del cuerpo del bucle mediante sentencias ajenas al paso regular dificulta enormemente el rastreo del algoritmo, pudiendo adelantar la salida del bucle o provocar saltos impredecibles en el número de iteraciones.

Criterios de elección

La decisión entre utilizar **while** con lectura adelantada o **do-while** depende de la relación temporal que existe entre la obtención de los datos de entrada y la evaluación de la condición de terminación.

Cuando se utiliza la sentencia **while** para procesar flujos de datos guiados por un **valor centinela**, es indispensable aplicar la técnica de la **lectura adelantada**[25] (*priming read*) antes de alcanzar el encabezado del bucle. Esta asignación previa permite inicializar la variable de control de modo que la primera comprobación del bucle sea capaz de detectar si el primer dato introducido por el usuario es ya el propio valor centinela. Si este es el caso, la condición fallará de inmediato y el cuerpo (del bucle) no se ejecutará ninguna vez, evitando procesar o alterar el estado del programa con un valor de terminación no válido.

Por el contrario, existen escenarios algorítmicos donde la propia naturaleza de la tarea exige que la interacción o lectura del dato se realice **dentro** del propio proceso repetitivo. El ejemplo más representativo de este patrón es la **validación defensiva de datos de entrada**[26]. En la validación de entradas, el programa debe solicitar obligatoriamente un dato al usuario, leerlo desde el teclado y, a continuación, comprobar si cumple los requisitos de validez prefijados. Si el dato es incorrecto, el programa debe mostrar un mensaje de error y volver a solicitarlo de forma reiterada hasta que la entrada sea correcta.

---

[25] La lectura adelantada (*priming read*) es un patrón de diseño clásico en la programación estructurada que consiste en obtener el primer dato de una secuencia antes de evaluar la condición del bucle por primera vez, garantizando que la variable de control o condición adquiera un estado válido y fiable antes de entrar en la estructura iterativa.

[26] Que veremos en el tema 5.

Si intentáramos implementar un proceso de validación utilizando la sentencia **while**, nos veríamos obligados a duplicar la instrucción de petición y lectura del dato fuera y dentro del bucle:

```
  // Implementación redundante de validación de entrada con bucle while
(desaconsejada)
  System.out.print("Introduce una nota (0 a 10): ");
  double nota = teclado.nextDouble(); // Lectura adelantada
  while ( nota < 0d || nota > 10d ) {
      System.out.println("Error: La nota debe estar entre 0.0 y 10.0.");
      System.out.print("Introduce una nota (0 a 10): ");
      nota = teclado.nextDouble(); // Lectura duplicada
  }
```

El uso de la sentencia **do-while** elimina por completo esta redundancia de código. Puesto que la primera solicitud y lectura debe ocurrir necesariamente al menos una vez, ubicar la entrada de datos dentro del cuerpo del bucle permite que la lectura se realice de forma incondicional, evaluando la validez del dato en la condición post-test:

```
  // Implementación con validación de entrada en bucle do-while
  double nota;

  do {
      System.out.print("Introduce una nota (0 a 10): ");
      nota = teclado.nextDouble(); // Se ejecuta incondicionalmente al menos
una vez

      if ( nota < 0d || nota > 10d ) {
          System.out.println("Error: La nota introducida no está en el rango
permitido.");
      }
  } while ( nota < 0d || nota > 10d ); // Se repite MIENTRAS la nota siga
siendo inválida
```

En este diseño, la variable `nota` se declara fuera del bucle para garantizar que su ámbito (*scope*) sea accesible tanto dentro del cuerpo como en la expresión de control. La petición y la lectura ocurren en la primera pasada; si el usuario introduce un valor correcto, la condición se evalúa inmediatamente como falsa, permitiendo la salida limpia del bucle tras una única iteración.

Reglas metodológicas para el diseño de bucles seguros

Para garantizar que cualquier estructura repetitiva sea correcta y segura, el proceso de diseño debe supervisar tres principios:

1. **Principio de terminación finita:** Todo bucle debe diseñarse con la garantía de que alcanzará la condición de terminación en un número finito de pasos. En los bucles por contador hay que verificar el sentido de la actualización; en los bucles por condición, el cuerpo del bucle debe contener sentencias que modifiquen de manera efectiva el estado de la variable de control.
2. **Definición clara del ámbito de la variable de control:** La variable de control de un bucle debe utilizarse solo en el bucle para el que se declara; Java permite, cuando se usa **for**, que esto se haga en la propia cabecera de la estructura, lo qeu permite que la memoria de la variable sea liberada automáticamente al concluir el bucle. Si la variable va a ser utilizada tras la finalización del proceso repetitivo (como un acumulado o dato validado), debe declararse en el bloque superior antes de ingresar en la estructura.
3. **Identificación de la invariante del bucle:** En la fase de análisis, es altamente recomendable determinar la **invariante del bucle**, definida como una proposición lógica que se mantiene verdadera antes de iniciar el bucle, al término de cada iteración y al concluir la estructura. Formular la invariante permite verificar razonadamente que el bucle transforma progresivamente el estado inicial de los datos en la solución buscada sin introducir efectos laterales ni inconsistencias en memoria.

Depuración y verificación

Hasta este momento hemos aprendido a construir programas que pueden ser traducidos por el compilador y a organizar sus instrucciones mediante secuencias, selecciones y repeticiones. Hemos visto cómo expresar operaciones, cómo utilizar condiciones para modificar el flujo de control y cómo construir algoritmos que pueden ejecutar una misma serie de instrucciones varias veces. Sin embargo, que un programa compile sin mensajes de advertencia o error representa una condición necesaria, pero en absoluto suficiente, para garantizar el éxito de la aplicación. Un programa cuyo código fuente cumple escrupulosamente con todas las reglas léxicas y sintácticas del lenguaje Java puede, sin embargo, detener su ejecución de manera abrupta o que permanezca ejecutándose indefinidamente. También puede suceder que el programa funcione correctamente para unos datos de prueba y falle cuando recibe otros. Por ello, la construcción de un programa no termina cuando conseguimos que el compilador lo acepte: debemos comprobar que su comportamiento es el que pretendíamos y, cuando no lo sea, averiguar por qué.

Esta parte del desarrollo recibe dos nombres relacionados pero no equivalentes: la **verificación** y la **depuración**. Mientras que la verificación abarca el conjunto de técnicas orientadas a evaluar si un programa satisface sus requisitos y produce los resultados esperados bajo cualquier escenario posible, la depuración (conocida universalmente en la jerga informática por el término inglés *debugging*[27]) comprende el proceso activo y sistemático mediante el cual el programador localiza la causa de una anomalía, aísla el fallo en el código fuente y corrige la lógica para que el programa tenga un comportamiento correcto. Por consiguiente, la verificación y la depuración no deben contemplarse como actividades improvisadas o accesorias al término del desarrollo: debe formar parte de todas las fases del diseño[28].

Verificación: Proceso mediante el cual comprobamos que un programa produce los resultados y el comportamiento esperados.

Depuración: Proceso de localizar y corregir las causas de los errores detectados en un programa o en el algoritmo que lo sustenta.

Error de sintaxis: Infracción de las reglas gramaticales del lenguaje de programación que impide la traducción del código fuente a código objeto por parte del compilador.

Error lógico o semántico: Anomalía en el diseño del algoritmo que permite la compilación y ejecución del programa pero genera resultados incorrectos o no deseados.

Error de ejecución: Fallo producido durante la ejecución del programa debido al intento de realizar una operación no permitida, provocando la interrupción de la aplicación.

Podemos entender fácilmente la relación entre ambas actividades si pensamos en un problema cotidiano. Cuando comprobamos que un aparato realiza correctamente la tarea para la que ha sido diseñado, estamos verificando su funcionamiento. Si descubrimos que no lo hace y buscamos qué componente está provocando el problema para corregirlo, estamos depurándolo.

En programación sucede algo semejante. Primero debemos preguntarnos **qué debería hacer el programa** y después observar **qué hace realmente**. Si ambos comportamientos coinciden, podremos considerar que la solución ha superado las pruebas realizadas. Si no coinciden, debemos determinar en qué punto se produce la desviación.

Consideremos, por ejemplo, el siguiente programa:

```
int numero = 1;
int suma = 0;
```

---

[27] El término *bug* ya era utilizado en el siglo XIX para referirse a pequeños fallos pero es en 1947 cuando la almirante Grace Hopper lo utilizó, tal vez como una broma, al descubrir que un mal funcionamiento en el Mark II era causado por una polilla; desde entonces el término *debugging* se asocia a identificar y corregir errores de programación.

[28] *Las pruebas de software pueden demostrar la presencia de errores, pero nunca su ausencia.* –Edsger W. Dijkstra.

```
while ( numero <= 5 ) {
    suma += numero;
    numero++;
}

System.out.println(suma);
```

A simple vista puede parecer evidente que el resultado será 15. Sin embargo, para asegurarnos de que el programa es correcto debemos ser capaces de justificar ese resultado y comprobar qué sucede en cada una de las iteraciones. Debemos saber qué valor tiene `numero` al entrar en el bucle, qué valor adquiere después de cada iteración, qué sucede con `suma` y en qué momento deja de cumplirse la condición.

La necesidad de realizar este análisis no desaparece cuando el programa funciona aparentemente bien. De hecho, una de las capacidades que debemos adquirir como programadores es la de **no confiar únicamente en una ejecución que haya producido el resultado esperado para un caso concreto**.

Supongamos que hemos escrito un programa que calcula el precio de un producto y lo probamos introduciendo un valor positivo. Si obtenemos el resultado esperado, no sabemos qué sucedería si el usuario introduce cero o un valor negativo, por ejemplo. Analogamente, un bucle que funciona correctamente cuando debe realizar diez iteraciones puede contener un error que solo aparezca cuando tenga que realizar cero, una o cien.

La verificación debe considerar, por tanto, **las diferentes situaciones que puede encontrar el programa**, no únicamente el caso que nos resulte más cómodo para comprobarlo.

Algunos errores son detectados directamente por las herramientas que utilizamos para escribir y traducir el programa. Otros solo aparecen cuando intentamos ejecutar determinadas instrucciones. Y otros, que son especialmente importantes, no producen ningún mensaje de error porque el programa es perfectamente válido desde el punto de vista del lenguaje, pero realiza una operación distinta de la que pretendíamos. Por tanto, no debemos confundir **«el programa no tiene errores»** con **«el programa es correcto»**.

Para abordar la depuración de manera eficiente, el primer paso consiste en categorizar la naturaleza de los fallos atendiendo al momento en el que son detectados y la entidad encargada de su identificación:

1. **Errores de compilación:** El código fuente vulnera la gramática formal del lenguaje. Estos errores son detectados de forma automática por el compilador (en Java, *javac*) antes de generar el código objeto. Aunque puedan resultar frustrantes para el principiante, son los más fáciles de subsanar.

2. **Errores de ejecución (*runtime errors*):** El programa ha superado con éxito la compilación y empieza a ejecutarse pero, en un momento dado, intenta realizar una instrucción inviable o prohibida. Un ejemplo clásico es la división entre cero.
3. **Errores lógicos o semánticos:** Son los más complejos. Un programa con un error lógico es plenamente válido para el compilador: no genera fallos de sintaxis ni interrumpe su ejecución. Sin embargo, la secuencia de operaciones conduce a resultados erróneos, selecciones incorrectas o bucles que nunca concluyen. Puesto que las computadoras se limitan a ejecutar ciegamente las órdenes recibidas, estos errores sólo pueden sacarse a la luz mediante el análisis crítico, la ejecución de baterías de prueba contrastadas y el rastreo minucioso del estado interno de las variables durante la ejecución.

Por esta razón, durante la programación:

- Primero, analizamos el problema y construimos una solución.
- Después, la codificamos en el lenguaje de programación.
- A continuación, comprobamos su comportamiento. Si descubrimos una discrepancia, buscamos la causa y modificamos el código; y, si es necesario, revisamos el propio algoritmo.

La posibilidad de volver atrás es una característica normal del desarrollo de software. Esta última situación es especialmente importante durante el aprendizaje. Programar no consiste exclusivamente en aprender la sintaxis del lenguaje. Debemos aprender a **razonar sobre el comportamiento del algoritmo** y a comprobar de manera sistemática que ese comportamiento coincide con el que exige el problema.

Por ello, verificar un programa significa algo más que ejecutarlo y observar si aparentemente «sale bien». Debemos elegir datos que nos permitan poner a prueba las distintas partes de la solución, conocer de antemano cuál debería ser el resultado y comparar ese resultado con el obtenido. Esto puede hacerse observando directamente la salida de la aplicación. En otras, necesitaremos conocer qué está ocurriendo con las variables durante la ejecución. Hay un amplio abanico de herramientas que va desde las técnicas manuales de comprobación en papel hasta el empleo de sofisticados depuradores interactivos (*debuggers*) integrados en los entornos de desarrollo (IDE). Entre estas técnicas destaca la **traza de ejecución**, un procedimiento sistemático que nos permite reconstruir paso a paso la ejecución del programa.

Una traza permite seguir el flujo de control del programa y registrar los cambios relevantes que se producen durante su ejecución. De este modo podemos averiguar no solo qué resultado obtiene el programa, sino también **cómo ha llegado hasta él**. Con estructuras repetitivas, una misma secuencia de sentencias puede ejecutarse muchas veces, pero los valores de las variables pueden ser diferentes en cada iteración. En el caso de las estructuras de selección, si el programa sigue un camino diferente del esperado, la traza permite determinar qué condición se evaluó, qué resultado produjo y qué alternativa fue ejecutada.

En definitiva, la verificación nos permite responder a la pregunta **«¿el programa hace lo que debería hacer?»**, mientras que la depuración nos obliga a responder a una pregunta más difícil cuando la respuesta es negativa: **«¿por qué no lo hace?»**.

Para responder a esta segunda pregunta debemos ser capaces de observar el programa durante su ejecución, seguir el flujo de control y conocer el estado de las variables en los distintos puntos del algoritmo. Por ello, antes de estudiar los errores que podemos encontrar en nuestros programas, veremos cómo realizar una **traza de ejecución**.

Trazas de ejecución

Una **traza de ejecución** —denominada también tradicionalmente en la metodología de la programación **prueba de escritorio** o seguimiento manual— es una de las técnicas más sencillas y, al mismo tiempo, más útiles para verificar el funcionamiento de un algoritmo que consiste, como su nombre indica, en realizar un **recorrido** de la ejecución. Es una técnica analítica fundamental que consiste en simular el comportamiento del programa ejecutando paso a paso un algoritmo sobre papel o mediante una herramienta automatizada. El objetivo no es únicamente comprobar cuál es el resultado que muestra el programa al terminar, sino registrar la evolución del **flujo de control**. De esta forma podemos determinar en qué momento una variable adquiere un valor incorrecto, qué condición se evalúa de manera distinta de lo esperado o qué sentencias se ejecutan un número de veces diferente al previsto.

Durante el aprendizaje de la programación, la traza de ejecución constituye la herramienta pedagógica más potente para desarrollar el modelo mental correcto sobre cómo un ordenador procesa el código. Es una representación ordenada de la ejecución de un programa. Para realizarla debemos partir del estado inicial de los datos e ir aplicando, en el mismo orden en que lo haría el ordenador, las instrucciones que forman el programa.

Traza de ejecución: Registro ordenado y cronológico de los estados sucesivos que adopta la memoria de un programa durante la ejecución paso a paso de sus instrucciones.

Prueba de escritorio: Método manual de verificación en el que el desarrollador simula la ejecución de un algoritmo siguiendo sus instrucciones sobre papel con un conjunto de datos de entrada representativos.

Estado del programa: Conjunto de valores que contienen todas las variables, constantes y estructuras de datos en un instante concreto del tiempo de ejecución.

Para construir una traza de ejecución de forma rigurosa y libre de ambigüedades, se utiliza una **tabla de traza**. En esta representación tabular, las columnas se organizan divididas en bloques funcionales:

1. **Paso:** Opcionalmente, el número de instrucción ejecutada.
2. **Sentencia:** Columna que indica el número de línea del código fuente o sentencia que se está ejecutando en ese preciso instante.

3. **Estado de la memoria (Variables):** Una o más columnas dedicadas a cada variable declarada en el programa, en la que se anota el valor que pasa a almacenar la variable cuando la instrucción en curso modifica su contenido.
4. **Resultado:** Columna auxiliar destinada a registrar el resultado: mensajes o datos que el programa imprime por la salida estándar.

Seguimiento de la ejecución

Veamos un programa sencillo; por ejemplo:

```
int a = 5;
int b = 3;
int resultado = a + b;
System.out.println(resultado);
```

Al comenzar, no existe ninguna de las variables utilizadas en el programa. La primera sentencia crea `a` y le asigna el valor `5`. La segunda crea `b` con el valor `3`. A continuación se evalúa la expresión `a + b`, cuyo resultado es `8`, y este valor se almacena en `resultado`. Finalmente, se muestra ese valor. Podemos representar esta ejecución mediante la tabla:

Paso	Sentencia	`a`	`b`	`resultado`
1	`int a = 5;`	5	—	—
2	`int b = 3;`	5	3	—
3	`int resultado = a + b;`	5	3	8
4	`System.out.println(resultado);`	5	3	8

El símbolo «—» indica que en ese momento la variable todavía no existe o no dispone de un valor que podamos utilizar. La tabla permite observar algo que el resultado final por sí solo no muestra: el valor de cada variable después de cada sentencia que modifica el estado del programa.

Trazas y flujo de control

Las trazas adquieren especial importancia cuando el programa contiene *estructuras de selección*. En estos casos, no todas las sentencias del programa se ejecutan. El flujo de control depende del resultado de una expresión lógica.

Por ejemplo:

```
int edad = 20;

if ( edad >= 18 )
    System.out.println("Mayor de edad");
else
    System.out.println("Menor de edad");
System.out.println("Fin");
```

Para realizar la traza debemos seguir el mismo proceso que seguiría el ordenador. Primero se crea la variable `edad` con el valor `20`. Después se evalúa la condición: `edad >= 18`. El resultado es `true`, por lo que se ejecutan las sentencias correspondientes a la rama `sí` y no las de la rama `else`.

La traza podría quedar así:

Paso	Ejecución	edad	Resultado de la condición
1	`int edad = 20;`	20	—
2	`edad >= 18`	20	`true`
3	`System.out.println(“Mayor de edad”);`	20	`true`
4	`System.out.println(“Fin”);`	20	`true`

La sentencia situada en el bloque `else` no aparece como paso ejecutado porque *no forma parte del recorrido seguido por el flujo de control* en esta ejecución concreta.

Esto es importante porque una traza no debe confundirse con una lectura lineal del código. El programa puede contener muchas sentencias, pero en una ejecución determinada solo se recorren aquellas que corresponden al camino seguido por el flujo de control. Podemos pensar, por tanto, que una traza responde en cada instante a dos preguntas: ¿Qué sentencia se está ejecutando? ¿Cuál es el estado del programa en ese momento? En un programa con selección, además, debemos prestar atención a *qué condiciones se evalúan y qué resultado producen*.

Trazas de estructuras repetitivas

Para ilustrar de forma práctica la elaboración de una traza sobre un algoritmo con estructuras repetitivas y operaciones numéricas, consideremos el siguiente fragmento escrito en Java, orientado a calcular el Máximo Común Divisor (MCD) de dos números enteros positivos mediante el algoritmo clásico de Euclides:

```
int a = 12;
int b = 18;

while ( b != 0 ) {
    int resto = a % b;
    a = b;
    b = resto;
}

System.out.println("MCD = " + a);
```

La tabla de traza resultante adopta la siguiente estructura:

Paso	Sentencia	a	b	resto	Condición (b != 0)	Salida estándar
1	Declaración a	12	—	—	—	—
2	Declaración b	12	18	—	—	—
3	Eval. **`while`**	12	18	—	18 != 0 → `true`	—
4	`resto = a % b`	12	18	12	—	—
5	`a = b`	18	18	12	—	—
6	`b = resto`	18	12	12	—	—
7	Eval. **`while`**	18	12	12	12 != 0 → `true`	—
8	`resto = a % b`	18	12	6	—	—
9	`a = b`	12	12	6	—	—
10	`b = resto`	12	6	6	—	—
11	Eval. **`while`**	12	6	6	6 != 0 → `true`	—
12	`resto = a % b`	12	6	0	—	—
13	`a = b`	6	6	0	—	—
14	`b = resto`	6	0	0	—	—
15	Eval. **`while`**	6	0	0	0 != 0 → `false`	—
16	Impresión final	6	0	0	—	`MCD = 6`

El método de Euclides divide el número mayor (dividendo) por el menor (divisor), obteniendo un cociente y un resto. Si es resto es cero, el divisor es el MCD; si es distinto de cero, el menor (divisor) pasa a ser el mayor (dividendo) y el resto pasa a ser el menor (divisor), repitiendo hasta que el resto es cero.

En el ejemplo, no ordenamos los números pero el algoritmo se corrige solo porque el cociente es cero, lo que provoca que el divisor (`18`) pase a ser el dividendo y el resto (`12`) pase a ser el divisor. En cada iteración, el resto reduce progresivamente el valor de `b`, garantizando que el algoritmo converja matemáticamente hacia el caso base. En la cuarta comprobación de la condición (`b != 0`), el valor de `b` ha pasado a ser `0`: la expresión evalúa a `false`, lo que provoca la salida del bucle. El resultado final (`6`) queda almacenado en `a`, permitiendo que la sentencia posterior imprima la respuesta correcta.

Reglas metodológicas para la realización de trazas efectivas

Para que una traza de ejecución cumpla eficazmente su función, es preciso asegurar:

1. **Selección de los juegos de prueba:** Una traza realizada con datos de entrada triviales puede ocultar errores. Es indispensable diseñar datos de prueba que incluyan **casos normales** (valores estándar), **casos límite o de frontera** (como valores nulos, cero o extremos de rangos numéricos) y **casos de error** (datos anómalos o fuera de rango) para comprobar la robustez de las validaciones defensivas.
2. **Fidelidad al código:** El diseñador debe abstenerse de presuponer lo que el programa “debería hacer” y limitarse a anotar estrictamente lo que las instrucciones escritas especifican. Asumir mentalmente que una variable ha sido

actualizada cuando la instrucción explícita de incremento no figura en el código es la causa principal de que los errores lógicos pasen desapercibidos durante la prueba.

3. **Actualización de variables:** En cada paso de la tabla, solo debe anotarse el valor de las variables que hayan sufrido una modificación explícita en la línea de código evaluada. Dejar las celdas en blanco o repetir el valor anterior permite identificar de un solo vistazo la frecuencia real de cambio de cada variable en memoria.

Errores comunes

A lo largo del aprendizaje de programación los programas aumentan su complejidad lo que implica que aumente la posibilidad de cometer errores. El alumnado se enfrenta de forma recurrente a una serie de fallos característicos que no se deben, en su mayoría, a una falta de comprensión teórica de los problemas, sino a la discrepancia entre el lenguaje natural —implícito, ambiguo y flexible— y la rigurosidad sintáctica y semántica que exige un lenguaje de programación. Una parte importante del aprendizaje de la programación consiste precisamente en aprender a reconocerlos, localizar su origen y corregirlos.

No todos los errores tienen la misma naturaleza ni se manifiestan del mismo modo. Como ya hemos comentado, algunos son detectados por el compilador (error sintáctico); otros aparecen durante la ejecución; y otros, los más difíciles de encontrar, no producen ningún mensaje de error y hacen que el programa obtenga un resultado diferente del esperado (error semántico).

Cuando el compilador muestra un error, debemos revisar primero la sintaxis y las restricciones del lenguaje. Cuando el programa se detiene durante la ejecución, debemos estudiar la operación que estaba realizando en ese momento. Y cuando termina normalmente pero el resultado no es el correcto, una traza de ejecución puede ayudarnos a descubrir dónde comenzó a desviarse el comportamiento respecto del esperado.

Errores de sintaxis y compilación

Uno de los errores sintácticos con mayor impacto lógico consiste en insertar un punto y coma tras la condición de control. Al colocar un punto y coma justo después de los paréntesis de un **if**, **while** o **for**, el compilador interpreta que el cuerpo supeditado a la condición es la *sentencia nula*:

```
// ERROR: El punto y coma anula el bloque condicional
if ( nota >= 5.0 );
    System.out.println("Aprobado");

// ERROR: Bucle infinito sobre la sentencia nula
while ( contador < 10 );
    contador++;
```

En la selección, la sentencia entre llaves se ejecutará siempre incondicionalmente. En la repetición, si la condición es verdadera, el programa entrará en un bucle infinito atrapado en la sentencia vacía.

Los errores más sencillos de detectar son aquellos que impiden compilar el programa. El compilador conoce las reglas sintácticas de Java y puede comprobar que las instrucciones están correctamente construidas. Entre los más habituales encontramos la omisión de un punto y coma, el uso incorrecto de paréntesis o llaves, una palabra reservada escrita en un lugar en el que se esperaba un identificador o una expresión que no cumple las reglas del lenguaje. Por ejemplo:

```
int numero = 10
System.out.println(numero);
```

La primera sentencia carece del punto y coma que debe cerrarla. El programa no puede llegar a ejecutarse porque la compilación se detiene al encontrar el problema. Otro caso frecuente es el de utilizar una variable que no ha sido declarada:

```
numero = 10;
System.out.println(numero);
```

En este caso el problema tampoco se encuentra durante la ejecución. El compilador no conoce ninguna variable denominada `numero` y, por tanto, no puede construir un programa válido.

Aunque menos, también es habitual durante el aprendizaje confundir las reglas sintácticas con las reglas de significado. Que una construcción tenga una forma aparentemente correcta no garantiza que Java permita utilizarla en ese contexto. Los tipos de datos forman parte de estas restricciones. Por ejemplo:

```
int numero = 5;
boolean mayor = numero;
```

La sintaxis de ambas declaraciones es reconocible, pero la segunda asignación intenta almacenar un valor de tipo **`int`** en una variable de tipo **`boolean`**. El compilador detectará que los tipos no son compatibles.

Algo similar es la confusión entre el operador de asignación (=) y el de igualdad (==): intentar evaluar una condición lógica utilizando la asignación en lugar de la comparación . Por ejmplo:

```
int x = 5;
if ( x = 10 )
   x += 4;
```

Mientras existen otros lenguajes en los que esto produce un fallo lógico, la fuerte tipificación de Java provoca un error porque el tipo no es booleano: sucede al intentar emplear valores numéricos (o de otro tipo) dentro de la condición de una estructura de control. Java exige de forma estricta que la expresión de control evalúe a un tipo primitivo `boolean` (`true` o `false`).

También, no asignar un valor inicial a variables locales es un error: el compilador detecta el riesgo de lectura de memoria no inicializada y detiene el proceso de compilación. Explicaremos este caso con detalle en el tema siguiente.

Cuando se produce un error de compilación conviene leer con atención el mensaje proporcionado por el compilador. El mensaje no siempre identifica exactamente la causa real del problema: en ocasiones señala el lugar en el que ha dejado de poder interpretar correctamente el código, que puede encontrarse después del punto en el que se introdujo el error. Aun así, proporciona una información valiosa para localizarlo.

Errores de ejecución

Se producen cuando la aplicación compila correctamente pero la máquina virtual interrumpe la ejecución al intentar procesar una operación no válida. Un ejemplo sencillo aparece en las operaciones aritméticas. Una división entera por cero no es una operación válida:

```
int dividendo = 10;
int divisor = 0;
int resultado = dividendo / divisor;
```

El programa es sintácticamente correcto y sus tipos son compatibles, pero al ejecutar la división se produce un error.

Muchos errores de ejecución tienen una característica especialmente importante: **el mismo programa puede ejecutarse correctamente para unos datos y fallar para otros**. Por ello, comprobar un único caso de prueba no garantiza que el programa esté preparado para cualquier entrada.

Errores en expresiones

Las expresiones son una fuente frecuente de errores lógicos porque una expresión puede ser completamente válida para el lenguaje y, aun así, no representar lo que el programador pretendía calcular. Uno de los errores más habituales consiste en olvidar las *reglas de precedencia* de los operadores. Por ejemplo:

```
int resultado = 10 + 5 * 2;
```

El resultado es `20`, porque la multiplicación tiene mayor precedencia que la suma. Si nuestra intención era sumar primero `10` y `5` y multiplicar después, tendremos que utilizar paréntesis:

```
int resultado = (10 + 5) * 2;
```

No existe ningún error de compilación en la primera versión. El problema está en que la expresión válida no representa el cálculo que pretendíamos realizar. Otro error frecuente aparece al mezclar valores enteros y reales sin tener en cuenta las reglas de conversión. En particular, debemos recordar que la división entre dos enteros produce un resultado entero:

```
int a = 5;
int b = 2;
double resultado = a / b;
```

El valor almacenado en `resultado` será `2.0`, no `2.5`. La división se ha realizado entre dos operandos de tipo entero antes de asignar el resultado al **`double`**. Si necesitamos realizar una división real debemos conseguir que alguno de los operandos participe como un valor de tipo real:

```
double resultado = (double) a / b;
```

El error, por tanto, no está necesariamente en la conversión final, sino en no haber tenido en cuenta el tipo de los operandos en el momento de evaluar la expresión. También puede producirse pérdida de información cuando utilizamos una conversión explícita. La conversión de un valor real a un tipo entero elimina su parte fraccionaria:

```
double numero = 7.8;
int entero = (int) numero;
```

En este caso `entero` tendrá el valor `7`. El *cast* no redondea el número.

Errores en condiciones

Las condiciones controlan el flujo de ejecución y, por tanto, un error en una condición puede cambiar completamente el comportamiento del programa. Un error muy frecuente consiste en escribir una comparación diferente de la que exige el problema. Por ejemplo, si queremos procesar los números comprendidos entre `1` y `10`, debemos determinar cuidadosamente si los extremos forman parte del intervalo:

```
numero >= 1 && numero <= 10
```

no representa exactamente lo mismo que:

```
numero > 1 && numero < 10
```

La diferencia se encuentra únicamente en los operadores de comparación, pero provoca que los valores `1` y `10` sean tratados de forma distinta.

Otro error habitual consiste en invertir la condición de una estructura de control. Cuando esto sucede, el programa sigue una rama diferente de la prevista y puede parecer que «ignora» determinadas instrucciones. Las trazas permiten detectar estos errores de forma bastante directa. Basta con anotar la condición evaluada y comprobar si su resultado coincide con el que debería producirse para los datos utilizados.

También debemos prestar especial atención a las expresiones lógicas formadas por varios operadores. Una expresión como:

```
edad >= 18 && tienePermiso || esInvitado
```

debe interpretarse teniendo en cuenta la precedencia de `&&` y `||`. Cuando una condición es compleja y puede resultar difícil de interpretar, es preferible utilizar paréntesis para hacer explícita la intención del algoritmo:

```
(edad >= 18 && tienePermiso) || esInvitado
```

Los paréntesis no siempre son necesarios desde el punto de vista sintáctico, pero sí pueden serlo desde el punto de vista de la claridad y de la prevención de errores.

Errores en estructuras de selección

En las estructuras **if-else** es frecuente escribir correctamente cada bloque y, sin embargo, asociar una sentencia al bloque equivocado. La indentación ayuda a visualizar esta estructura, pero debemos recordar que los espacios y tabulaciones no determinan el ámbito de las sentencias en Java. Por ello, cuando un bloque contiene más de una sentencia, debemos utilizar llaves de forma explícita:

```
if ( condición ) {
    sentencia1;
    sentencia2;
}
```

También es posible que una cadena de decisiones **if-else-if** contenga condiciones que se solapen. En ese caso, puede existir una condición posterior que nunca llegue a evaluarse porque una condición anterior ya resulta verdadera. Cuando esto sucede, la estructura es válida para Java, pero puede ser incorrecta desde el punto de vista del algoritmo. Conviene revisar siempre si las condiciones están ordenadas y construidas de forma que todas las situaciones que deben distinguirse puedan alcanzarse.

Análogamente puede producirse una asociación errónea de *else* huérfano. El desarrollador puede indentar el código intentando vincular el **else** con el primer **if**, pero la regla gramatical de Java lo asocia invariablemente con el **if** más cercano:

```
// Código engañoso debido a la sangría
if ( saldo > 0 )
    if ( importe <= saldo )
        saldo -= importe;
else
    System.out.println("Saldo insuficiente"); // ¡Se asocia al segundo 'if', no al primero!
```

En una sentencia **switch**, el error más característico consiste en olvidar que, en su forma normal, una vez encontrada una etiqueta **`case`**, la ejecución continúa con las sentencias siguientes hasta abandonar el bloque (o hasta encontrar un **`break`**).

Por ejemplo:

```
switch (dia) {
    case 1:
        System.out.println("Lunes");
    case 2:
        System.out.println("Martes");
        break;
}
```

Si dia vale `1`, se mostrará «Lunes» y, a continuación, también «Martes». Esto no es un error del lenguaje: es el comportamiento definido por la sentencia. El error aparece cuando este comportamiento no era el que pretendíamos.

Por esta razón, cuando una sentencia **switch** no produce el resultado esperado, debemos comprobar cuidadosamente el camino seguido por cada **case** y la presencia de las sentencias **break** allí donde sean necesarias.

Errores en bucles

Los bucles concentran algunos de los errores más habituales durante el aprendizaje de la programación. El problema puede estar en la condición, en la inicialización, en la actualización de la variable de control o en la relación entre estos elementos. Uno de los errores más graves es diseñar un bucle infinito:

```
int numero = 1;

while ( numero <= 10 )
    System.out.println(numero);
```

La condición depende de `numero`, pero ninguna sentencia del cuerpo modifica su valor. Como consecuencia, si `numero` comienza siendo `1`, la condición continuará siendo verdadera indefinidamente. Un error de este tipo puede detectarse mediante una traza. Si construimos varias iteraciones y observamos que el estado relevante no cambia de manera que permita alcanzar la condición de terminación, el problema resulta evidente.

También es frecuente actualizar la variable de control en el sentido contrario al necesario. Por ejemplo:

```
for ( int numero = 1; numero <= 10; numero -= 1 )
    System.out.println(numero);
```

La condición exige que `numero` aumente hasta superar `10`, pero la actualización hace que disminuya. En este caso el bucle tampoco podrá alcanzar la condición de terminación prevista.

Otro error clásico consiste en utilizar mal los límites del bucle, provocando que se ejecute una iteración de más o de menos. Este problema se conoce habitualmente como **desviación en uno** o *off-by-one*. Por ejemplo, para mostrar los números del `1` al `10`, estas dos condiciones no son equivalentes:

```
numero <= 10
```

```
numero < 10
```

La segunda excluye el valor `10`. La diferencia puede parecer pequeña, pero en algoritmos más complejos puede alterar completamente el resultado. Para evitar estos errores es conveniente preguntarse siempre, antes de escribir el bucle, cuál debe ser el primer valor tratado, cuál debe ser el último y qué debe suceder inmediatamente después de procesar ese último valor.

También puede producirse el error contrario: utilizar un bucle **while** cuando el cuerpo debe ejecutarse necesariamente al menos una vez. En estas situaciones, un **do-while** representa de forma más natural el algoritmo y evita tener que duplicar la lectura o la inicialización fuera de la estructura.

Confundir el valor de una variable con su actualización

Cuando se utiliza una variable como contador o acumulador, debemos distinguir claramente entre el valor que contiene y la operación que modifica dicho valor. Por ejemplo:

```
suma = suma + numero;
```

no significa que `suma` tome simplemente el valor de `numero`. Su nuevo valor es el resultado de combinar el valor anterior de `suma` con el de `numero`. Si escribimos por error:

```
suma = numero;
```

el valor acumulado anterior se pierde en cada iteración. Las trazas resultan especialmente útiles para descubrir este tipo de errores, porque permiten observar si una variable que debería conservar y ampliar progresivamente su información está siendo sobrescrita en lugar de actualizada.

Algo semejante sucede con los operadores de incremento y decremento. En una sentencia independiente:

```
numero++;
```

el efecto final es incrementar `numero` en una unidad. Pero cuando el operador forma parte de una expresión, la diferencia entre su utilización prefija y posfija puede modificar el valor utilizado por el resto de la expresión. Por ello, cuando aparezca un `++` o un `--` dentro de una expresión, debemos seguir cuidadosamente el orden de evaluación.

Errores que el compilador no puede detectar

Existe una idea fundamental que debemos recordar: el compilador puede comprobar las reglas de Java, pero no puede conocer nuestra intención. Si escribimos:

```
int precio = 100;
int descuento = 20;
int resultado = precio + descuento;
```

no existe ningún motivo para que Java rechace el programa. Todas las sentencias son correctas y los tipos son compatibles. Sin embargo, si lo que queríamos calcular era el precio después de aplicar un descuento del veinte por ciento, el algoritmo es incorrecto.

Este tipo de error no puede solucionarse simplemente leyendo los mensajes del compilador. Debemos comprobar el comportamiento del programa y compararlo con lo que establece el problema. Aquí adquieren toda su importancia las **trazas de ejecución y los casos de prueba**. Una traza nos permite reconstruir qué ha hecho realmente el programa; los casos de prueba nos permiten comprobar si ese comportamiento coincide con el que debería producirse para diferentes situaciones.

Estrategia general para localizar fallos

Cuando un programa no funciona correctamente, no resulta eficaz modificar el código al azar hasta que aparentemente produzca el resultado esperado. Es preferible seguir un proceso ordenado.

En primer lugar, determinar qué comportamiento esperábamos y compararlo con el que se ha producido realmente. A partir de esa diferencia podemos decidir dónde comenzar la investigación.

Si el programa no compila, debemos revisar el código en busca de errores sintácticos, de tipos o de uso incorrecto de las construcciones del lenguaje.

Si compila pero se detiene durante la ejecución, debemos estudiar la operación que se estaba realizando en el momento del fallo y los valores de los datos que intervenían en ella.

Si termina sin producir errores pero ofrece un resultado incorrecto, debemos realizar una traza, siguiendo el flujo de control y anotando la evolución de las variables relevantes. El punto que debemos buscar es la primera decisión o modificación que se aparta del comportamiento esperado. Una vez localizada, debemos preguntarnos si el problema se encuentra en la implementación del algoritmo o en el propio algoritmo. No siempre basta con modificar una sentencia: en ocasiones la solución exige revisar el planteamiento que condujo a ella. Esta forma de trabajar es especialmente importante en los bucles y en las estructuras de selección, donde una pequeña modificación puede alterar todo el recorrido de la ejecución. Cambiar una condición porque «parece que funciona» puede ocultar el problema en lugar de resolverlo.

La programación no consiste únicamente en conseguir que un programa compile o produzca una salida para un caso concreto. Un programa correcto debe respetar las reglas del lenguaje, comportarse de forma adecuada durante su ejecución y, sobre todo, implementar el algoritmo que resuelve el problema planteado.

Aprender a localizar errores forma parte, por tanto, del propio proceso de programación. La escritura del código, su ejecución, la realización de pruebas, las trazas y la corrección de los errores no son actividades independientes, sino partes de un mismo proceso de construcción y verificación de programas.